

File Tree

해시 확인: certutil -hashfile <dist> SHA256

dist/

├─ index.html (77433 B / 8ab941e91919)
├─ main.js (33823 B / 04a8e344941e)
└─ style.css (14083 B / 3bb92984b4f4)

==== index.html (77433 B / 8ab941e91919) ====

```
1 | <!doctype html>
2 | <html lang="ko">
3 |   <head>
4 |     <meta charset="UTF-8" />
5 |     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6 |     <title>TS-Chess</title>
7 |     <script id="ts-chess-shared-core-source" type="text/plain">
8 |       var TSChessShared = (function (exports) {
9 |         'use strict';
10 |         const MASK64 = 0xffffffffffffffff;
11 |         function rotl(x, k) {
12 |           return ((x << k) | (x >> (64n - k))) & 0xffffffffffffffff;
13 |         }
14 |         function wrappingMul(x, y) {
15 |           return (x * y) & MASK64;
16 |         }
17 |         function xoroshiro128(state) {
18 |           return function () {
19 |             let s0 = BigInt(state & MASK64);
20 |             let s1 = BigInt((state >> 64n) & MASK64);
21 |             const result = wrappingMul(rotl(wrappingMul(s0, 5n), 7n), 9n);
22 |             s1 ^= s0;
23 |             s0 = (rotl(s0, 24n) ^ s1 ^ (s1 << 16n)) & MASK64;
24 |             s1 = rotl(s1, 37n);
25 |             state = (s1 << 64n) | s0;
26 |             return result;
27 |           };
28 |         }
29 |         const rand = xoroshiro128(0xa187eb39cdcaed8f31c4b365b102e01en);
30 |         const PIECE_KEYS = Array.from({ length: 2 }, () =>
31 |           Array.from({ length: 6 }, () => Array.from({ length: 128 }, () => rand()))
32 |         );
33 |         const EP_KEYS = Array.from({ length: 8 }, () => rand());
34 |         const CASTLING_KEYS = Array.from({ length: 16 }, () => rand());
35 |         const SIDE_KEY = rand();
36 |         const WHITE = 'w';
37 |         const BLACK = 'b';
38 |         const PAWN = 'p';
39 |         const KNIGHT = 'n';
40 |         const BISHOP = 'b';
41 |         const ROOK = 'r';
42 |         const QUEEN = 'q';
43 |         const KING = 'k';
44 |         const DEFAULT_POSITION = 'rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/RNBQKBNR w KQkq -- 0 1';
45 |         class Move {
46 |           color;
47 |           from;
48 |           to;
49 |           piece;
50 |           captured;
51 |           promotion;
52 |           /**
53 |            * @deprecated This field is deprecated and will be removed in version 2.0.0.
54 |            * Please use move descriptor functions instead: `isCapture`, `isPromotion`,
55 |            * `isEnPassant`, `isKingsideCastle`, `isQueensideCastle`, `isCastle`, and
56 |            * `isBigPawn`
57 |            */
58 |           flags;
59 |           san;
60 |           lan;
61 |           before;
```

```

62 |         after;
63 |         constructor(chess, internal) {
64 |             const { color, piece, from, to, flags, captured, promotion } = internal;
65 |             const fromAlgebraic = algebraic(from);
66 |             const toAlgebraic = algebraic(to);
67 |             this.color = color;
68 |             this.piece = piece;
69 |             this.from = fromAlgebraic;
70 |             this.to = toAlgebraic;
71 |             this.san = chess['_moveToSan'](internal, chess['_moves']({ legal: true }));
72 |             this.lan = fromAlgebraic + toAlgebraic;
73 |             this.before = chess.fen();
74 |             chess['_makeMove'](internal);
75 |             this.after = chess.fen();
76 |             chess['_undoMove']();
77 |             this.flags = '';
78 |             for (const flag in BITS) {
79 |                 if (BITS[flag] & flags) {
80 |                     this.flags += FLAGS[flag];
81 |                 }
82 |             }
83 |             if (captured) {
84 |                 this.captured = captured;
85 |             }
86 |             if (promotion) {
87 |                 this.promotion = promotion;
88 |                 this.lan += promotion;
89 |             }
90 |         }
91 |         isCapture() {
92 |             return this.flags.indexOf(FLAGS['CAPTURE']) > -1;
93 |         }
94 |         isPromotion() {
95 |             return this.flags.indexOf(FLAGS['PROMOTION']) > -1;
96 |         }
97 |         isEnPassant() {
98 |             return this.flags.indexOf(FLAGS['EP_CAPTURE']) > -1;
99 |         }
100 |         isKingsideCastle() {
101 |             return this.flags.indexOf(FLAGS['KSIDE_CASTLE']) > -1;
102 |         }
103 |         isQueensideCastle() {
104 |             return this.flags.indexOf(FLAGS['QSIDE_CASTLE']) > -1;
105 |         }
106 |         isBigPawn() {
107 |             return this.flags.indexOf(FLAGS['BIG_PAWN']) > -1;
108 |         }
109 |     }
110 |     const EMPTY = -1;
111 |     const FLAGS = {
112 |         NORMAL: 'n',
113 |         CAPTURE: 'c',
114 |         BIG_PAWN: 'b',
115 |         EP_CAPTURE: 'e',
116 |         PROMOTION: 'p',
117 |         KSIDE_CASTLE: 'k',
118 |         QSIDE_CASTLE: 'q',
119 |         NULL_MOVE: '-',
120 |     };
121 |     const SQUARES = [
122 |         'a8',
123 |         'b8',
124 |         'c8',
125 |         'd8',
126 |         'e8',
127 |         'f8',
128 |         'g8',
129 |         'h8',
130 |         'a7',
131 |         'b7',
132 |         'c7',

```

```

133 | | | | | 'd7',
134 | | | | | 'e7',
135 | | | | | 'f7',
136 | | | | | 'g7',
137 | | | | | 'h7',
138 | | | | | 'a6',
139 | | | | | 'b6',
140 | | | | | 'c6',
141 | | | | | 'd6',
142 | | | | | 'e6',
143 | | | | | 'f6',
144 | | | | | 'g6',
145 | | | | | 'h6',
146 | | | | | 'a5',
147 | | | | | 'b5',
148 | | | | | 'c5',
149 | | | | | 'd5',
150 | | | | | 'e5',
151 | | | | | 'f5',
152 | | | | | 'g5',
153 | | | | | 'h5',
154 | | | | | 'a4',
155 | | | | | 'b4',
156 | | | | | 'c4',
157 | | | | | 'd4',
158 | | | | | 'e4',
159 | | | | | 'f4',
160 | | | | | 'g4',
161 | | | | | 'h4',
162 | | | | | 'a3',
163 | | | | | 'b3',
164 | | | | | 'c3',
165 | | | | | 'd3',
166 | | | | | 'e3',
167 | | | | | 'f3',
168 | | | | | 'g3',
169 | | | | | 'h3',
170 | | | | | 'a2',
171 | | | | | 'b2',
172 | | | | | 'c2',
173 | | | | | 'd2',
174 | | | | | 'e2',
175 | | | | | 'f2',
176 | | | | | 'g2',
177 | | | | | 'h2',
178 | | | | | 'a1',
179 | | | | | 'b1',
180 | | | | | 'c1',
181 | | | | | 'd1',
182 | | | | | 'e1',
183 | | | | | 'f1',
184 | | | | | 'g1',
185 | | | | | 'h1',
186 | | | | | ];
187 | | | | | const BITS = {
188 | | | | |     NORMAL: 1,
189 | | | | |     CAPTURE: 2,
190 | | | | |     BIG_PAWN: 4,
191 | | | | |     EP_CAPTURE: 8,
192 | | | | |     PROMOTION: 16,
193 | | | | |     KSIDE_CASTLE: 32,
194 | | | | |     QSIDE_CASTLE: 64,
195 | | | | |     NULL_MOVE: 128,
196 | | | | | };
197 | | | | | const 0x88 = {
198 | | | | |     a8: 0,
199 | | | | |     b8: 1,
200 | | | | |     c8: 2,
201 | | | | |     d8: 3,
202 | | | | |     e8: 4,
203 | | | | |     f8: 5,

```

```

204 | | | | g8: 6,
205 | | | | h8: 7,
206 | | | | a7: 16,
207 | | | | b7: 17,
208 | | | | c7: 18,
209 | | | | d7: 19,
210 | | | | e7: 20,
211 | | | | f7: 21,
212 | | | | g7: 22,
213 | | | | h7: 23,
214 | | | | a6: 32,
215 | | | | b6: 33,
216 | | | | c6: 34,
217 | | | | d6: 35,
218 | | | | e6: 36,
219 | | | | f6: 37,
220 | | | | g6: 38,
221 | | | | h6: 39,
222 | | | | a5: 48,
223 | | | | b5: 49,
224 | | | | c5: 50,
225 | | | | d5: 51,
226 | | | | e5: 52,
227 | | | | f5: 53,
228 | | | | g5: 54,
229 | | | | h5: 55,
230 | | | | a4: 64,
231 | | | | b4: 65,
232 | | | | c4: 66,
233 | | | | d4: 67,
234 | | | | e4: 68,
235 | | | | f4: 69,
236 | | | | g4: 70,
237 | | | | h4: 71,
238 | | | | a3: 80,
239 | | | | b3: 81,
240 | | | | c3: 82,
241 | | | | d3: 83,
242 | | | | e3: 84,
243 | | | | f3: 85,
244 | | | | g3: 86,
245 | | | | h3: 87,
246 | | | | a2: 96,
247 | | | | b2: 97,
248 | | | | c2: 98,
249 | | | | d2: 99,
250 | | | | e2: 100,
251 | | | | f2: 101,
252 | | | | g2: 102,
253 | | | | h2: 103,
254 | | | | a1: 112,
255 | | | | b1: 113,
256 | | | | c1: 114,
257 | | | | d1: 115,
258 | | | | e1: 116,
259 | | | | f1: 117,
260 | | | | g1: 118,
261 | | | | h1: 119,
262 | | | | };
263 | | | | const PAWN_OFFSETS = {
264 | | | | b: [16, 32, 17, 15],
265 | | | | w: [-16, -32, -17, -15],
266 | | | | };
267 | | | | const PIECE_OFFSETS = {
268 | | | | n: [-18, -33, -31, -14, 18, 33, 31, 14],
269 | | | | b: [-17, -15, 17, 15],
270 | | | | r: [-16, 1, 16, -1],
271 | | | | q: [-17, -16, -15, 1, 17, 16, 15, -1],
272 | | | | k: [-17, -16, -15, 1, 17, 16, 15, -1],
273 | | | | };
274 | | | | const ATTACKS = [

```

```

275 | | | | 20, 0, 0, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 0, 20, 0, 0, 20, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 20,
276 | | | | 0, 0, 0, 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 0, 20, 0, 0, 0, 0, 20, 0, 0, 0, 24, 0, 0, 0, 20,
277 | | | | 0, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0, 20, 0, 0, 0, 0, 0, 0, 0, 20, 2, 24, 2, 20,
278 | | | | 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0, 0, 24, 24, 24, 24, 24, 24, 56,
279 | | | | 0, 56, 24, 24, 24, 24, 24, 24, 0, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
280 | | | | 0, 20, 2, 24, 2, 20, 0, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0, 20, 0, 0, 0, 0, 0, 0, 0,
281 | | | | 0, 20, 0, 0, 0, 24, 0, 0, 0, 20, 0, 0, 0, 0, 20, 0, 0, 0, 24, 0, 0, 0, 0, 20, 0, 0, 0,
282 | | | | 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 20, 0, 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 0, 0,
283 | | | | 20,
284 | | | | ];
285 | | | | const RAYS = [
286 | | | | 17, 0, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 0, 15, 0, 0, 17, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 0, 15,
287 | | | | 0, 0, 0, 0, 17, 0, 0, 0, 0, 16, 0, 0, 0, 0, 15, 0, 0, 0, 0, 0, 0, 17, 0, 0, 0, 16, 0, 0, 0, 15,
288 | | | | 0, 0, 0, 0, 0, 0, 0, 0, 17, 0, 0, 16, 0, 0, 15, 0, 0, 0, 0, 0, 0, 0, 0, 17, 0, 16, 0, 15,
289 | | | | 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 17, 16, 15, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, -1,
290 | | | | -1, -1, -1, -1, -1, -1, 0, 0, 0, 0, 0, 0, 0, 0, -15, -16, -17, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
291 | | | | -15, 0, -16, 0, -17, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, -15, 0, 0, -16, 0, 0, -17, 0, 0, 0, 0, 0, 0,
292 | | | | 0, 0, -15, 0, 0, 0, -16, 0, 0, 0, -17, 0, 0, 0, 0, 0, 0, -15, 0, 0, 0, 0, -16, 0, 0, 0, 0, -17,
293 | | | | 0, 0, 0, 0, -15, 0, 0, 0, 0, 0, -16, 0, 0, 0, 0, 0, -17, 0, 0, -15, 0, 0, 0, 0, 0, 0, -16, 0, 0,
294 | | | | 0, 0, 0, 0, -17,
295 | | | | ];
296 | | | | const PIECE_MASKS = { p: 1, n: 2, b: 4, r: 8, q: 16, k: 32 };
297 | | | | const SYMBOLS = 'pnbrqkPNBRQK';
298 | | | | const PROMOTIONS = [KNIGHT, BISHOP, ROOK, QUEEN];
299 | | | | const RANK_1 = 7;
300 | | | | const RANK_2 = 6;
301 | | | | const RANK_7 = 1;
302 | | | | const RANK_8 = 0;
303 | | | | const ROOKS = {
304 | | | |   w: [
305 | | | |     { square: 0x88.a1, flag: BITS.QSIDE_CASTLE },
306 | | | |     { square: 0x88.h1, flag: BITS.KSIDE_CASTLE },
307 | | | |   ],
308 | | | |   b: [
309 | | | |     { square: 0x88.a8, flag: BITS.QSIDE_CASTLE },
310 | | | |     { square: 0x88.h8, flag: BITS.KSIDE_CASTLE },
311 | | | |   ],
312 | | | | };
313 | | | | const SECOND_RANK = { b: RANK_7, w: RANK_2 };
314 | | | | function rank(square) {
315 | | | |   return square >> 4;
316 | | | | }
317 | | | | function file(square) {
318 | | | |   return square & 15;
319 | | | | }
320 | | | | function isDigit(c) {
321 | | | |   return '0123456789'.indexOf(c) !== -1;
322 | | | | }
323 | | | | function algebraic(square) {
324 | | | |   const f = file(square);
325 | | | |   const r = rank(square);
326 | | | |   return 'abcdefgh'.substring(f, f + 1) + '87654321'.substring(r, r + 1);
327 | | | | }
328 | | | | function swapColor(color) {
329 | | | |   return color === WHITE ? BLACK : WHITE;
330 | | | | }
331 | | | | function validateFen(fen) {
332 | | | |   const tokens = fen.split(/\s+/);
333 | | | |   if (tokens.length !== 6) {
334 | | | |     return {
335 | | | |       ok: false,
336 | | | |       error: 'Invalid FEN: must contain six space-delimited fields',
337 | | | |     };
338 | | | |   }
339 | | | |   const moveNumber = parseInt(tokens[5], 10);
340 | | | |   if (isNaN(moveNumber) || moveNumber <= 0) {
341 | | | |     return {
342 | | | |       ok: false,
343 | | | |       error: 'Invalid FEN: move number must be a positive integer',
344 | | | |     };
345 | | | |   }

```

```

346 |         const halfMoves = parseInt(tokens[4], 10);
347 |         if (isNaN(halfMoves) || halfMoves < 0) {
348 |             return {
349 |                 ok: false,
350 |                 error: 'Invalid FEN: half move counter number must be a non-negative integer',
351 |             };
352 |         }
353 |         if (!/^(-?[abcdefgh][36])$/.test(tokens[3])) {
354 |             return { ok: false, error: 'Invalid FEN: en-passant square is invalid' };
355 |         }
356 |         if (/^[^kKqQ-]/.test(tokens[2])) {
357 |             return { ok: false, error: 'Invalid FEN: castling availability is invalid' };
358 |         }
359 |         if (!/^(w|b)$/.test(tokens[1])) {
360 |             return { ok: false, error: 'Invalid FEN: side-to-move is invalid' };
361 |         }
362 |         const rows = tokens[0].split('/');
363 |         if (rows.length !== 8) {
364 |             return {
365 |                 ok: false,
366 |                 error: 'Invalid FEN: piece data does not contain 8 '/'-delimited rows',
367 |             };
368 |         }
369 |         for (let i = 0; i < rows.length; i++) {
370 |             let sumFields = 0;
371 |             let previousWasNumber = false;
372 |             for (let k = 0; k < rows[i].length; k++) {
373 |                 if (isDigit(rows[i][k])) {
374 |                     if (previousWasNumber) {
375 |                         return {
376 |                             ok: false,
377 |                             error: 'Invalid FEN: piece data is invalid (consecutive number)',
378 |                         };
379 |                     }
380 |                     sumFields += parseInt(rows[i][k], 10);
381 |                     previousWasNumber = true;
382 |                 } else {
383 |                     if (!/^[prnbqkPRNBQK]$/.test(rows[i][k])) {
384 |                         return {
385 |                             ok: false,
386 |                             error: 'Invalid FEN: piece data is invalid (invalid piece)',
387 |                         };
388 |                     }
389 |                     sumFields += 1;
390 |                     previousWasNumber = false;
391 |                 }
392 |             }
393 |             if (sumFields !== 8) {
394 |                 return {
395 |                     ok: false,
396 |                     error: 'Invalid FEN: piece data is invalid (too many squares in rank)',
397 |                 };
398 |             }
399 |         }
400 |         if ((tokens[3][1] === '3' & tokens[1] === 'w') || (tokens[3][1] === '6' & tokens[1] === 'b')) {
401 |             return { ok: false, error: 'Invalid FEN: illegal en-passant square' };
402 |         }
403 |         const kings = [
404 |             { color: 'white', regex: /K/g },
405 |             { color: 'black', regex: /k/g },
406 |         ];
407 |         for (const { color, regex } of kings) {
408 |             if (!regex.test(tokens[0])) {
409 |                 return { ok: false, error: `Invalid FEN: missing ${color} king` };
410 |             }
411 |             if ((tokens[0].match(regex) || []).length > 1) {
412 |                 return { ok: false, error: `Invalid FEN: too many ${color} kings` };
413 |             }
414 |         }
415 |         if (Array.from(rows[0] + rows[7]).some((char) => char.toUpperCase() === 'P')) {
416 |             return {

```

```

417 |         ok: false,
418 |         error: 'Invalid FEN: some pawns are on the edge rows',
419 |     });
420 |     }
421 |     return { ok: true };
422 | }
423 | function getDisambiguator(move, moves) {
424 |     const from = move.from;
425 |     const to = move.to;
426 |     const piece = move.piece;
427 |     let ambiguities = 0;
428 |     let sameRank = 0;
429 |     let sameFile = 0;
430 |     for (let i = 0, len = moves.length; i < len; i++) {
431 |         const ambigFrom = moves[i].from;
432 |         const ambigTo = moves[i].to;
433 |         const ambigPiece = moves[i].piece;
434 |         if (piece === ambigPiece && from !== ambigFrom && to === ambigTo) {
435 |             ambiguities++;
436 |             if (rank(from) === rank(ambigFrom)) {
437 |                 sameRank++;
438 |             }
439 |             if (file(from) === file(ambigFrom)) {
440 |                 sameFile++;
441 |             }
442 |         }
443 |     }
444 |     if (ambiguities > 0) {
445 |         if (sameRank > 0 && sameFile > 0) {
446 |             return algebraic(from);
447 |         } else if (sameFile > 0) {
448 |             return algebraic(from).charAt(1);
449 |         } else {
450 |             return algebraic(from).charAt(0);
451 |         }
452 |     }
453 |     return '';
454 | }
455 | function addMove(moves, color, from, to, piece, captured = void 0, flags = BITS.NORMAL) {
456 |     const r = rank(to);
457 |     if (piece === PAWN && (r === RANK_1 || r === RANK_8)) {
458 |         for (let i = 0; i < PROMOTIONS.length; i++) {
459 |             const promotion = PROMOTIONS[i];
460 |             moves.push({
461 |                 color,
462 |                 from,
463 |                 to,
464 |                 piece,
465 |                 captured,
466 |                 promotion,
467 |                 flags: flags | BITS.PROMOTION,
468 |             });
469 |         }
470 |     } else {
471 |         moves.push({
472 |             color,
473 |             from,
474 |             to,
475 |             piece,
476 |             captured,
477 |             flags,
478 |         });
479 |     }
480 | }
481 | class Chess {
482 |     _board = new Array(128);
483 |     _turn = WHITE;
484 |     _kings = { w: EMPTY, b: EMPTY };
485 |     _epSquare = -1;
486 |     _halfMoves = 0;
487 |     _moveNumber = 0;

```

```

488 |         _history = [];
489 |         _castling = { w: 0, b: 0 };
490 |         _hash = 0n;
491 |         // tracks number of times a position has been seen for repetition checking
492 |         _positionCount = /* @__PURE__ */ new Map();
493 |         constructor(fen = DEFAULT_POSITION, { skipValidation = false } = {}) {
494 |             this.load(fen, { skipValidation });
495 |         }
496 |         clear() {
497 |             this._board = new Array(128);
498 |             this._kings = { w: EMPTY, b: EMPTY };
499 |             this._turn = WHITE;
500 |             this._castling = { w: 0, b: 0 };
501 |             this._epSquare = EMPTY;
502 |             this._halfMoves = 0;
503 |             this._moveNumber = 1;
504 |             this._history = [];
505 |             this._hash = this._computeHash();
506 |             this._positionCount = /* @__PURE__ */ new Map();
507 |         }
508 |         load(fen, { skipValidation = false } = {}) {
509 |             let tokens = fen.split(/\s+/);
510 |             if (tokens.length ≥ 2 && tokens.length < 6) {
511 |                 const adjustments = ['-', '-', '0', '1'];
512 |                 fen = tokens.concat(adjustments.slice(-(6 - tokens.length))).join(' ');
513 |             }
514 |             tokens = fen.split(/\s+/);
515 |             if (!skipValidation) {
516 |                 const { ok, error } = validateFen(fen);
517 |                 if (!ok) {
518 |                     throw new Error(error);
519 |                 }
520 |             }
521 |             const position = tokens[0];
522 |             let square = 0;
523 |             this.clear();
524 |             for (let i = 0; i < position.length; i++) {
525 |                 const piece = position.charAt(i);
526 |                 if (piece === '/') {
527 |                     square += 8;
528 |                 } else if (isDigit(piece)) {
529 |                     square += parseInt(piece, 10);
530 |                 } else {
531 |                     const color = piece < 'a' ? WHITE : BLACK;
532 |                     this._put({ type: piece.toLowerCase(), color }, algebraic(square));
533 |                     square++;
534 |                 }
535 |             }
536 |             this._turn = tokens[1];
537 |             if (tokens[2].indexOf('K') > -1) {
538 |                 this._castling.w |= BITS.KSIDE_CASTLE;
539 |             }
540 |             if (tokens[2].indexOf('Q') > -1) {
541 |                 this._castling.w |= BITS.QSIDE_CASTLE;
542 |             }
543 |             if (tokens[2].indexOf('k') > -1) {
544 |                 this._castling.b |= BITS.KSIDE_CASTLE;
545 |             }
546 |             if (tokens[2].indexOf('q') > -1) {
547 |                 this._castling.b |= BITS.QSIDE_CASTLE;
548 |             }
549 |             this._epSquare = tokens[3] === '-' ? EMPTY : 0x88[tokens[3]];
550 |             this._halfMoves = parseInt(tokens[4], 10);
551 |             this._moveNumber = parseInt(tokens[5], 10);
552 |             this._hash = this._computeHash();
553 |             this._incPositionCount();
554 |         }
555 |         fen({ forceEnpassantSquare = false } = {}) {
556 |             let empty = 0;
557 |             let fen = '';
558 |             for (let i = 0x88.a8; i ≤ 0x88.h1; i++) {

```



```

559 |         if (this._board[i]) {
560 |             if (empty > 0) {
561 |                 fen += empty;
562 |                 empty = 0;
563 |             }
564 |             const { color, type: piece } = this._board[i];
565 |             fen += color === WHITE ? piece.toUpperCase() : piece.toLowerCase();
566 |         } else {
567 |             empty++;
568 |         }
569 |         if ((i + 1) & 136) {
570 |             if (empty > 0) {
571 |                 fen += empty;
572 |             }
573 |             if (i !== 0x88.h1) {
574 |                 fen += '/';
575 |             }
576 |             empty = 0;
577 |             i += 8;
578 |         }
579 |     }
580 |     let castling = '';
581 |     if (this._castling[WHITE] & BITS.KSIDE_CASTLE) {
582 |         castling += 'K';
583 |     }
584 |     if (this._castling[WHITE] & BITS.QSIDE_CASTLE) {
585 |         castling += 'Q';
586 |     }
587 |     if (this._castling[BLACK] & BITS.KSIDE_CASTLE) {
588 |         castling += 'k';
589 |     }
590 |     if (this._castling[BLACK] & BITS.QSIDE_CASTLE) {
591 |         castling += 'q';
592 |     }
593 |     castling = castling || '-';
594 |     let epSquare = '-';
595 |     if (this._epSquare !== EMPTY) {
596 |         if (forceEnpassantSquare) {
597 |             epSquare = algebraic(this._epSquare);
598 |         } else {
599 |             const bigPawnSquare = this._epSquare + (this._turn === WHITE ? 16 : -16);
600 |             const squares = [bigPawnSquare + 1, bigPawnSquare - 1];
601 |             for (const square of squares) {
602 |                 if (square & 136) {
603 |                     continue;
604 |                 }
605 |                 const color = this._turn;
606 |                 if (this._board[square]?.color === color & this._board[square]?.type === PAWN) {
607 |                     this._makeMove({
608 |                         color,
609 |                         from: square,
610 |                         to: this._epSquare,
611 |                         piece: PAWN,
612 |                         captured: PAWN,
613 |                         flags: BITS.EP_CAPTURE,
614 |                     });
615 |                     const isLegal = !this._isKingAttacked(color);
616 |                     this._undoMove();
617 |                     if (isLegal) {
618 |                         epSquare = algebraic(this._epSquare);
619 |                         break;
620 |                     }
621 |                 }
622 |             }
623 |         }
624 |     }
625 |     return [fen, this._turn, castling, epSquare, this._halfMoves, this._moveNumber].join(' ');
626 | }
627 | _pieceKey(i) {
628 |     if (!this._board[i]) {
629 |         return 0n;

```

```

630 |         }
631 |         const { color, type } = this._board[i];
632 |         const colorIndex = {
633 |             w: 0,
634 |             b: 1,
635 |         }[color];
636 |         const typeIndex = {
637 |             p: 0,
638 |             n: 1,
639 |             b: 2,
640 |             r: 3,
641 |             q: 4,
642 |             k: 5,
643 |         }[type];
644 |         return PIECE_KEYS[colorIndex][typeIndex][i];
645 |     }
646 |     _epKey() {
647 |         return this._epSquare === EMPTY ? 0n : EP_KEYS[this._epSquare & 7];
648 |     }
649 |     _castlingKey() {
650 |         const index = (this._castling.w >> 5) | (this._castling.b >> 3);
651 |         return CASTLING_KEYS[index];
652 |     }
653 |     _computeHash() {
654 |         let hash = 0n;
655 |         for (let i = 0x88.a8; i <= 0x88.h1; i++) {
656 |             if (i & 136) {
657 |                 i += 7;
658 |                 continue;
659 |             }
660 |             if (this._board[i]) {
661 |                 hash ^= this._pieceKey(i);
662 |             }
663 |         }
664 |         hash ^= this._epKey();
665 |         hash ^= this._castlingKey();
666 |         if (this._turn === 'b') {
667 |             hash ^= SIDE_KEY;
668 |         }
669 |         return hash;
670 |     }
671 |     reset() {
672 |         this.load(DEFAULT_POSITION);
673 |     }
674 |     get(square) {
675 |         return this._board[0x88[square]];
676 |     }
677 |     _set(sq, piece) {
678 |         this._hash ^= this._pieceKey(sq);
679 |         this._board[sq] = piece;
680 |         this._hash ^= this._pieceKey(sq);
681 |     }
682 |     _put({ type, color }, square) {
683 |         if (SYMBOLS.indexOf(type.toLowerCase()) === -1) {
684 |             return false;
685 |         }
686 |         if (!(square in 0x88)) {
687 |             return false;
688 |         }
689 |         const sq = 0x88[square];
690 |         if (type === KING & !(this._kings[color] === EMPTY || this._kings[color] === sq)) {
691 |             return false;
692 |         }
693 |         const currentPieceOnSquare = this._board[sq];
694 |         if (currentPieceOnSquare & currentPieceOnSquare.type === KING) {
695 |             this._kings[currentPieceOnSquare.color] = EMPTY;
696 |         }
697 |         this._set(sq, { type, color });
698 |         if (type === KING) {
699 |             this._kings[color] = sq;
700 |         }

```

```

701 |         return true;
702 |     }
703 |     _clear(sq) {
704 |         this._hash ^= this._pieceKey(sq);
705 |         delete this._board[sq];
706 |     }
707 |     _attacked(color, square, verbose) {
708 |         const attackers = [];
709 |         for (let i = 0x88.a8; i ≤ 0x88.h1; i++) {
710 |             if (i & 136) {
711 |                 i += 7;
712 |                 continue;
713 |             }
714 |             if (this._board[i] === void 0 || this._board[i].color !== color) {
715 |                 continue;
716 |             }
717 |             const piece = this._board[i];
718 |             const difference = i - square;
719 |             if (difference === 0) {
720 |                 continue;
721 |             }
722 |             const index = difference + 119;
723 |             if (ATTACKS[index] & PIECE_MASKS[piece.type]) {
724 |                 if (piece.type === PAWN) {
725 |                     if (
726 |                         (difference > 0 && piece.color === WHITE) ||
727 |                         (difference ≤ 0 && piece.color === BLACK)
728 |                     ) {
729 |                         if (!verbose) {
730 |                             return true;
731 |                         } else {
732 |                             attackers.push(algebraic(i));
733 |                         }
734 |                     }
735 |                     continue;
736 |                 }
737 |                 if (piece.type === 'n' || piece.type === 'k') {
738 |                     if (!verbose) {
739 |                         return true;
740 |                     } else {
741 |                         attackers.push(algebraic(i));
742 |                         continue;
743 |                     }
744 |                 }
745 |                 const offset = RAYS[index];
746 |                 let j = i + offset;
747 |                 let blocked = false;
748 |                 while (j !== square) {
749 |                     if (this._board[j] !== null) {
750 |                         blocked = true;
751 |                         break;
752 |                     }
753 |                     j += offset;
754 |                 }
755 |                 if (!blocked) {
756 |                     if (!verbose) {
757 |                         return true;
758 |                     } else {
759 |                         attackers.push(algebraic(i));
760 |                         continue;
761 |                     }
762 |                 }
763 |             }
764 |         }
765 |         if (verbose) {
766 |             return attackers;
767 |         } else {
768 |             return false;
769 |         }
770 |     }
771 |     attackers(square, attackedBy) {

```

```

772 |         if (!attackedBy) {
773 |             return this._attacked(this._turn, 0x88[square], true);
774 |         } else {
775 |             return this._attacked(attackedBy, 0x88[square], true);
776 |         }
777 |     }
778 |     _isKingAttacked(color) {
779 |         const square = this._kings[color];
780 |         return square === -1 ? false : this._attacked(swapColor(color), square);
781 |     }
782 |     hash() {
783 |         return this._hash.toString(16);
784 |     }
785 |     isAttacked(square, attackedBy) {
786 |         return this._attacked(attackedBy, 0x88[square]);
787 |     }
788 |     isCheck() {
789 |         return this._isKingAttacked(this._turn);
790 |     }
791 |     inCheck() {
792 |         return this.isCheck();
793 |     }
794 |     isCheckmate() {
795 |         return this.isCheck() && this._moves().length === 0;
796 |     }
797 |     isStalemate() {
798 |         return !this.isCheck() && this._moves().length === 0;
799 |     }
800 |     isInsufficientMaterial() {
801 |         const pieces = {
802 |             b: 0,
803 |             n: 0,
804 |             r: 0,
805 |             q: 0,
806 |             k: 0,
807 |             p: 0,
808 |         };
809 |         const bishops = [];
810 |         let numPieces = 0;
811 |         let squareColor = 0;
812 |         for (let i = 0x88.a8; i <= 0x88.h1; i++) {
813 |             squareColor = (squareColor + 1) % 2;
814 |             if (i & 136) {
815 |                 i += 7;
816 |                 continue;
817 |             }
818 |             const piece = this._board[i];
819 |             if (piece) {
820 |                 pieces[piece.type] = piece.type in pieces ? pieces[piece.type] + 1 : 1;
821 |                 if (piece.type === BISHOP) {
822 |                     bishops.push(squareColor);
823 |                 }
824 |                 numPieces++;
825 |             }
826 |         }
827 |         if (numPieces === 2) {
828 |             return true;
829 |         } else if (
830 |             // k vs. kn .... or .... k vs. kb
831 |             numPieces === 3 &&
832 |             (pieces[BISHOP] === 1 || pieces[KNIGHT] === 1)
833 |         ) {
834 |             return true;
835 |         } else if (numPieces === pieces[BISHOP] + 2) {
836 |             let sum = 0;
837 |             const len = bishops.length;
838 |             for (let i = 0; i < len; i++) {
839 |                 sum += bishops[i];
840 |             }
841 |             if (sum === 0 || sum === len) {
842 |                 return true;

```

```

843 |         }
844 |     }
845 |     return false;
846 | }
847 | isThreefoldRepetition() {
848 |     return this._getPositionCount(this._hash) ≥ 3;
849 | }
850 | isDrawByFiftyMoves() {
851 |     return this._halfMoves ≥ 100;
852 | }
853 | isDraw() {
854 |     return (
855 |         this.isDrawByFiftyMoves() ||
856 |         this.isStalemate() ||
857 |         this.isInsufficientMaterial() ||
858 |         this.isThreefoldRepetition()
859 |     );
860 | }
861 | isGameOver() {
862 |     return this.isCheckmate() || this.isDraw();
863 | }
864 | moves({ verbose = false, square = void 0, piece = void 0 } = {}) {
865 |     const moves = this._moves({ square, piece });
866 |     if (verbose) {
867 |         return moves.map((move) ⇒ new Move(this, move));
868 |     } else {
869 |         return moves.map((move) ⇒ this._moveToSan(move, moves));
870 |     }
871 | }
872 | _moves({ legal = true, piece = void 0, square = void 0 } = {}) {
873 |     const forSquare = square ? square.toLowerCase() : void 0;
874 |     const forPiece = piece ? piece.toLowerCase();
875 |     const moves = [];
876 |     const us = this._turn;
877 |     const them = swapColor(us);
878 |     let firstSquare = 0x88.a8;
879 |     let lastSquare = 0x88.h1;
880 |     let singleSquare = false;
881 |     if (forSquare) {
882 |         if (!(forSquare in 0x88)) {
883 |             return [];
884 |         } else {
885 |             firstSquare = lastSquare = 0x88[forSquare];
886 |             singleSquare = true;
887 |         }
888 |     }
889 |     for (let from = firstSquare; from ≤ lastSquare; from++) {
890 |         if (from & 136) {
891 |             from += 7;
892 |             continue;
893 |         }
894 |         if (!this._board[from] || this._board[from].color === them) {
895 |             continue;
896 |         }
897 |         const { type } = this._board[from];
898 |         let to;
899 |         if (type === PAWN) {
900 |             if (forPiece & forPiece !== type) continue;
901 |             to = from + PAWN_OFFSETS[us][0];
902 |             if (!this._board[to]) {
903 |                 addMove(moves, us, from, to, PAWN);
904 |                 to = from + PAWN_OFFSETS[us][1];
905 |                 if (SECOND_RANK[us] === rank(from) & !this._board[to]) {
906 |                     addMove(moves, us, from, to, PAWN, void 0, BITS.BIG_PAWN);
907 |                 }
908 |             }
909 |             for (let j = 2; j < 4; j++) {
910 |                 to = from + PAWN_OFFSETS[us][j];
911 |                 if (to & 136) continue;
912 |                 if (this._board[to] ? color === them) {
913 |                     addMove(moves, us, from, to, PAWN, this._board[to].type, BITS.CAPTURE);

```

```

914 |         } else if (to === this._epSquare) {
915 |             addMove(moves, us, from, to, PAWN, PAWN, BITS.EP_CAPTURE);
916 |         }
917 |     }
918 |     } else {
919 |         if (forPiece && forPiece !== type) continue;
920 |         for (let j = 0, len = PIECE_OFFSETS[type].length; j < len; j++) {
921 |             const offset = PIECE_OFFSETS[type][j];
922 |             to = from;
923 |             while (true) {
924 |                 to += offset;
925 |                 if (to & 136) break;
926 |                 if (!this._board[to]) {
927 |                     addMove(moves, us, from, to, type);
928 |                 } else {
929 |                     if (this._board[to].color === us) break;
930 |                     addMove(moves, us, from, to, type, this._board[to].type, BITS.CAPTURE);
931 |                     break;
932 |                 }
933 |                 if (type === KNIGHT || type === KING) break;
934 |             }
935 |         }
936 |     }
937 | }
938 | if (forPiece === void 0 || forPiece === KING) {
939 |     if (!singleSquare || lastSquare === this._kings[us]) {
940 |         if (this._castling[us] & BITS.KSIDE_CASTLE) {
941 |             const castlingFrom = this._kings[us];
942 |             const castlingTo = castlingFrom + 2;
943 |             if (
944 |                 !this._board[castlingFrom + 1] &&
945 |                 !this._board[castlingTo] &&
946 |                 !this._attacked(them, this._kings[us]) &&
947 |                 !this._attacked(them, castlingFrom + 1) &&
948 |                 !this._attacked(them, castlingTo)
949 |             ) {
950 |                 addMove(moves, us, this._kings[us], castlingTo, KING, void 0, BITS.KSIDE_CASTLE);
951 |             }
952 |         }
953 |         if (this._castling[us] & BITS.QSIDE_CASTLE) {
954 |             const castlingFrom = this._kings[us];
955 |             const castlingTo = castlingFrom - 2;
956 |             if (
957 |                 !this._board[castlingFrom - 1] &&
958 |                 !this._board[castlingFrom - 2] &&
959 |                 !this._board[castlingFrom - 3] &&
960 |                 !this._attacked(them, this._kings[us]) &&
961 |                 !this._attacked(them, castlingFrom - 1) &&
962 |                 !this._attacked(them, castlingTo)
963 |             ) {
964 |                 addMove(moves, us, this._kings[us], castlingTo, KING, void 0, BITS.QSIDE_CASTLE);
965 |             }
966 |         }
967 |     }
968 | }
969 | if (!legal || this._kings[us] === -1) {
970 |     return moves;
971 | }
972 | const legalMoves = [];
973 | for (let i = 0, len = moves.length; i < len; i++) {
974 |     this._makeMove(moves[i]);
975 |     if (!this._isKingAttacked(us)) {
976 |         legalMoves.push(moves[i]);
977 |     }
978 |     this._undoMove();
979 | }
980 | return legalMoves;
981 | }
982 | move(move) {
983 |     let moveObj = null;
984 |     const moves = this._moves();

```

```

985 |         for (let i = 0, len = moves.length; i < len; i++) {
986 |             if (
987 |                 move.from === algebraic(moves[i].from) &&
988 |                 move.to === algebraic(moves[i].to) &&
989 |                 (!('promotion' in moves[i]) || move.promotion === moves[i].promotion)
990 |             ) {
991 |                 moveObj = moves[i];
992 |                 break;
993 |             }
994 |         }
995 |         if (!moveObj) {
996 |             throw new Error(`Invalid move: ${JSON.stringify(move)}`);
997 |         }
998 |         const prettyMove = new Move(this, moveObj);
999 |         this._makeMove(moveObj);
1000 |         this._incPositionCount();
1001 |         return prettyMove;
1002 |     }
1003 |     _push(move) {
1004 |         this._history.push({
1005 |             move,
1006 |             kings: { b: this._kings.b, w: this._kings.w },
1007 |             turn: this._turn,
1008 |             castling: { b: this._castling.b, w: this._castling.w },
1009 |             epSquare: this._epSquare,
1010 |             halfMoves: this._halfMoves,
1011 |             moveNumber: this._moveNumber,
1012 |         });
1013 |     }
1014 |     _movePiece(from, to) {
1015 |         this._hash ^= this._pieceKey(from);
1016 |         this._board[to] = this._board[from];
1017 |         delete this._board[from];
1018 |         this._hash ^= this._pieceKey(to);
1019 |     }
1020 |     _makeMove(move) {
1021 |         const us = this._turn;
1022 |         const them = swapColor(us);
1023 |         this._push(move);
1024 |         if (move.flags & BITS.NULL_MOVE) {
1025 |             if (us === BLACK) {
1026 |                 this._moveNumber++;
1027 |             }
1028 |             this._halfMoves++;
1029 |             this._turn = them;
1030 |             this._epSquare = EMPTY;
1031 |             return;
1032 |         }
1033 |         this._hash ^= this._epKey();
1034 |         this._hash ^= this._castlingKey();
1035 |         if (move.captured) {
1036 |             this._hash ^= this._pieceKey(move.to);
1037 |         }
1038 |         this._movePiece(move.from, move.to);
1039 |         if (move.flags & BITS.EP_CAPTURE) {
1040 |             if (this._turn === BLACK) {
1041 |                 this._clear(move.to - 16);
1042 |             } else {
1043 |                 this._clear(move.to + 16);
1044 |             }
1045 |         }
1046 |         if (move.promotion) {
1047 |             this._clear(move.to);
1048 |             this._set(move.to, { type: move.promotion, color: us });
1049 |         }
1050 |         if (this._board[move.to].type === KING) {
1051 |             this._kings[us] = move.to;
1052 |             if (move.flags & BITS.KSIDE_CASTLE) {
1053 |                 const castlingTo = move.to - 1;
1054 |                 const castlingFrom = move.to + 1;
1055 |                 this._movePiece(castlingFrom, castlingTo);

```

```

1056 |         } else if (move.flags & BITS.QSIDE_CASTLE) {
1057 |             const castlingTo = move.to + 1;
1058 |             const castlingFrom = move.to - 2;
1059 |             this._movePiece(castlingFrom, castlingTo);
1060 |         }
1061 |         this._castling[us] = 0;
1062 |     }
1063 |     if (this._castling[us]) {
1064 |         for (let i = 0, len = ROOKS[us].length; i < len; i++) {
1065 |             if (move.from === ROOKS[us][i].square && this._castling[us] & ROOKS[us][i].flag) {
1066 |                 this._castling[us] ^= ROOKS[us][i].flag;
1067 |                 break;
1068 |             }
1069 |         }
1070 |     }
1071 |     if (this._castling[them]) {
1072 |         for (let i = 0, len = ROOKS[them].length; i < len; i++) {
1073 |             if (move.to === ROOKS[them][i].square && this._castling[them] & ROOKS[them][i].flag) {
1074 |                 this._castling[them] ^= ROOKS[them][i].flag;
1075 |                 break;
1076 |             }
1077 |         }
1078 |     }
1079 |     this._hash ^= this._castlingKey();
1080 |     if (move.flags & BITS.BIG_PAWN) {
1081 |         let epSquare;
1082 |         if (us === BLACK) {
1083 |             epSquare = move.to - 16;
1084 |         } else {
1085 |             epSquare = move.to + 16;
1086 |         }
1087 |         if (
1088 |             (((move.to - 1) & 136) &&
1089 |             this._board[move.to - 1]?.type === PAWN &&
1090 |             this._board[move.to - 1]?.color === them) ||
1091 |             (((move.to + 1) & 136) &&
1092 |             this._board[move.to + 1]?.type === PAWN &&
1093 |             this._board[move.to + 1]?.color === them)
1094 |         ) {
1095 |             this._epSquare = epSquare;
1096 |             this._hash ^= this._epKey();
1097 |         } else {
1098 |             this._epSquare = EMPTY;
1099 |         }
1100 |     } else {
1101 |         this._epSquare = EMPTY;
1102 |     }
1103 |     if (move.piece === PAWN) {
1104 |         this._halfMoves = 0;
1105 |     } else if (move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE)) {
1106 |         this._halfMoves = 0;
1107 |     } else {
1108 |         this._halfMoves++;
1109 |     }
1110 |     if (us === BLACK) {
1111 |         this._moveNumber++;
1112 |     }
1113 |     this._turn = them;
1114 |     this._hash ^= SIDE_KEY;
1115 | }
1116 | undo() {
1117 |     const hash = this._hash;
1118 |     const move = this._undoMove();
1119 |     if (move) {
1120 |         const prettyMove = new Move(this, move);
1121 |         this._decPositionCount(hash);
1122 |         return prettyMove;
1123 |     }
1124 |     return null;
1125 | }
1126 | _undoMove() {

```



```

1127 |         const old = this._history.pop();
1128 |         if (old === void 0) {
1129 |             return null;
1130 |         }
1131 |         this._hash ^= this._epKey();
1132 |         this._hash ^= this._castlingKey();
1133 |         const move = old.move;
1134 |         this._kings = old.kings;
1135 |         this._turn = old.turn;
1136 |         this._castling = old.castling;
1137 |         this._epSquare = old.epSquare;
1138 |         this._halfMoves = old.halfMoves;
1139 |         this._moveNumber = old.moveNumber;
1140 |         this._hash ^= this._epKey();
1141 |         this._hash ^= this._castlingKey();
1142 |         this._hash ^= SIDE_KEY;
1143 |         const us = this._turn;
1144 |         const them = swapColor(us);
1145 |         if (move.flags & BITS.NULL_MOVE) {
1146 |             return move;
1147 |         }
1148 |         this._movePiece(move.to, move.from);
1149 |         if (move.piece) {
1150 |             this._clear(move.from);
1151 |             this._set(move.from, { type: move.piece, color: us });
1152 |         }
1153 |         if (move.captured) {
1154 |             if (move.flags & BITS.EP_CAPTURE) {
1155 |                 let index;
1156 |                 if (us === BLACK) {
1157 |                     index = move.to - 16;
1158 |                 } else {
1159 |                     index = move.to + 16;
1160 |                 }
1161 |                 this._set(index, { type: PAWN, color: them });
1162 |             } else {
1163 |                 this._set(move.to, { type: move.captured, color: them });
1164 |             }
1165 |         }
1166 |         if (move.flags & (BITS.KSIDE_CASTLE | BITS.QSIDE_CASTLE)) {
1167 |             let castlingTo, castlingFrom;
1168 |             if (move.flags & BITS.KSIDE_CASTLE) {
1169 |                 castlingTo = move.to + 1;
1170 |                 castlingFrom = move.to - 1;
1171 |             } else {
1172 |                 castlingTo = move.to - 2;
1173 |                 castlingFrom = move.to + 1;
1174 |             }
1175 |             this._movePiece(castlingFrom, castlingTo);
1176 |         }
1177 |         return move;
1178 |     }
1179 |     // Convert a move from 0x88 coordinates to Standard Algebraic Notation.
1180 |     _moveToSan(move, moves) {
1181 |         let output = '';
1182 |         if (move.flags & BITS.KSIDE_CASTLE) {
1183 |             output = 'O-O';
1184 |         } else if (move.flags & BITS.QSIDE_CASTLE) {
1185 |             output = 'O-O-O';
1186 |         } else {
1187 |             if (move.piece === PAWN) {
1188 |                 const disambiguator = getDisambiguator(move, moves);
1189 |                 output += move.piece.toUpperCase() + disambiguator;
1190 |             }
1191 |             if (move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE)) {
1192 |                 if (move.piece === PAWN) {
1193 |                     output += algebraic(move.from)[0];
1194 |                 }
1195 |                 output += 'x';
1196 |             }
1197 |             output += algebraic(move.to);

```

```

1198 |         if (move.promotion) {
1199 |             output += '=' + move.promotion.toUpperCase();
1200 |         }
1201 |     }
1202 |     this._makeMove(move);
1203 |     if (this.isCheck()) {
1204 |         if (this.isCheckmate()) {
1205 |             output += '#';
1206 |         } else {
1207 |             output += '+';
1208 |         }
1209 |     }
1210 |     this._undoMove();
1211 |     return output;
1212 | }
1213 | turn() {
1214 |     return this._turn;
1215 | }
1216 | board() {
1217 |     const output = [];
1218 |     let row = [];
1219 |     for (let i = 0x88.a8; i ≤ 0x88.h1; i++) {
1220 |         if (this._board[i] === null) {
1221 |             row.push(null);
1222 |         } else {
1223 |             row.push({
1224 |                 square: algebraic(i),
1225 |                 type: this._board[i].type,
1226 |                 color: this._board[i].color,
1227 |             });
1228 |         }
1229 |         if ((i + 1) & 136) {
1230 |             output.push(row);
1231 |             row = [];
1232 |             i += 8;
1233 |         }
1234 |     }
1235 |     return output;
1236 | }
1237 | squareColor(square) {
1238 |     if (square in 0x88) {
1239 |         const sq = 0x88[square];
1240 |         return (rank(sq) + file(sq)) % 2 === 0 ? 'light' : 'dark';
1241 |     }
1242 |     return null;
1243 | }
1244 | history({ verbose = false } = {}) {
1245 |     const reversedHistory = [];
1246 |     const moveHistory = [];
1247 |     while (this._history.length > 0) {
1248 |         reversedHistory.push(this._undoMove());
1249 |     }
1250 |     while (true) {
1251 |         const move = reversedHistory.pop();
1252 |         if (!move) {
1253 |             break;
1254 |         }
1255 |         if (verbose) {
1256 |             moveHistory.push(new Move(this, move));
1257 |         } else {
1258 |             moveHistory.push(this._moveToSan(move, this._moves()));
1259 |         }
1260 |         this._makeMove(move);
1261 |     }
1262 |     return moveHistory;
1263 | }
1264 | /*
1265 |  * Keeps track of position occurrence counts for the purpose of repetition
1266 |  * checking. Old positions are removed from the map if their counts are reduced to 0.
1267 |  */
1268 | _getPositionCount(hash) {

```

```

1269 |         return this._positionCount.get(hash) ?? 0;
1270 |     }
1271 |     _incPositionCount() {
1272 |         this._positionCount.set(this._hash, (this._positionCount.get(this._hash) ?? 0) + 1);
1273 |     }
1274 |     _decPositionCount(hash) {
1275 |         const currentCount = this._positionCount.get(hash) ?? 0;
1276 |         if (currentCount === 1) {
1277 |             this._positionCount.delete(hash);
1278 |         } else {
1279 |             this._positionCount.set(hash, currentCount - 1);
1280 |         }
1281 |     }
1282 |     moveNumber() {
1283 |         return this._moveNumber;
1284 |     }
1285 | }
1286 | exports.BISHOP = BISHOP;
1287 | exports.BLACK = BLACK;
1288 | exports.Chess = Chess;
1289 | exports.DEFAULT_POSITION = DEFAULT_POSITION;
1290 | exports.KING = KING;
1291 | exports.KNIGHT = KNIGHT;
1292 | exports.Move = Move;
1293 | exports.PAWN = PAWN;
1294 | exports.QUEEN = QUEEN;
1295 | exports.ROOK = ROOK;
1296 | exports.SQUARES = SQUARES;
1297 | exports.WHITE = WHITE;
1298 | exports.validateFen = validateFen;
1299 | exports.xoroshiro128 = xoroshiro128;
1300 | Object.defineProperty(exports, Symbol.toStringTag, { value: 'Module' });
1301 | return exports;
1302 | })({});
1303 |
1304 | globalThis.TSChessShared = TSChessShared;
1305 | </script>
1306 |
1307 | <script id="ai-worker-source-1" type="text/plain">
1308 |     !(function () {
1309 |         'use strict';
1310 |         const sharedCore = globalThis.TSChessShared;
1311 |         if (!sharedCore) throw new Error('Missing shared chess runtime: TSChessShared');
1312 |         const Chess = sharedCore.Chess;
1313 |         sharedCore.SQUARES;
1314 |         const HARD_MAX_DEPTH = Number.POSITIVE_INFINITY,
1315 |               PIECE_VALUES = { p: 100, n: 320, b: 330, r: 500, q: 900, k: 0 },
1316 |               STARTING_NON_PAWN_MATERIAL =
1317 |                 2 * (PIECE_VALUES.q + 2 * PIECE_VALUES.r + 2 * PIECE_VALUES.b + 2 * PIECE_VALUES.n),
1318 |               OPENING_PHASE_END_MATERIAL = STARTING_NON_PAWN_MATERIAL / 2;
1319 |         function searchRoot(chess, depth, ctx, previousBestMove) {
1320 |             const candidates = [],
1321 |                   legalMoves = orderMoves(chess.moves({ verbose: !0 }), previousBestMove);
1322 |             for (const move of legalMoves) {
1323 |                 (checkTime(ctx), chess.move(toMoveCommand(move)));
1324 |                 const score = -negamax(chess, depth - 1, -1e7, 1e7, 1, ctx);
1325 |                 (chess.undo(), candidates.push({ move: toMoveCommand(move), san: move.san, score: score }));
1326 |             }
1327 |             if (0 === candidates.length) throw new SearchTimeout();
1328 |             const principal = (function selectBestRootMove(candidates) {
1329 |                 return candidates.reduce((currentBest, candidate) =>
1330 |                     candidate.score > currentBest.score ? candidate : currentBest,
1331 |                     {});
1332 |             })(candidates),
1333 |                   selection = (function selectRootMove(candidates, settings, best) {
1334 |                       const level = settings.randomness / 30,
1335 |                             windowCp = 20 + 150 * level,
1336 |                             floorScore = best.score - windowCp,
1337 |                             ticketPower = 4 - 1.5 * level;
1338 |                       if (
1339 |                           (function shouldForceBestMove(settings, best) {

```

```

1340 |         return (
1341 |             0 === settings.randomness ||
1342 |             (function isMateProtectedScore(score) {
1343 |                 return Math.abs(score) ≥ 99e4;
1344 |             })(best.score) ||
1345 |             ('hard' === settings.preset && best.score < 0)
1346 |         );
1347 |     })(settings, best)
1348 | )
1349 | return {
1350 |     selected: best,
1351 |     debug: createDeterministicDebug(
1352 |         candidates,
1353 |         best,
1354 |         best,
1355 |         windowCp,
1356 |         floorScore,
1357 |         ticketPower,
1358 |     ),
1359 | };
1360 | const ticketCandidates = (function selectTicketCandidates(candidates, settings, best) {
1361 |     return 'hard' === settings.preset && best.score > 0
1362 |     ? candidates.filter((candidate) => candidate.score > 0)
1363 |     : candidates;
1364 | })(candidates, settings, best),
1365 | weightedCandidates = ticketCandidates
1366 |     .map((candidate) => ({
1367 |         candidate: candidate,
1368 |         tickets: (Math.max(0, candidate.score - floorScore) / windowCp) ** ticketPower,
1369 |     })))
1370 |     .filter(({ tickets: tickets }) => tickets > 0),
1371 | totalTickets = weightedCandidates.reduce(
1372 |     (total, { tickets: tickets }) => total + tickets,
1373 |     0,
1374 | );
1375 | if (totalTickets ≤ 0)
1376 |     return {
1377 |         selected: best,
1378 |         debug: createDeterministicDebug(
1379 |             candidates,
1380 |             best,
1381 |             best,
1382 |             windowCp,
1383 |             floorScore,
1384 |             ticketPower,
1385 |         ),
1386 |     };
1387 | let draw = Math.random() * totalTickets,
1388 | selected = best;
1389 | for (const { candidate: candidate, tickets: tickets } of weightedCandidates)
1390 |     if (((draw -= tickets), draw ≤ 0)) {
1391 |         selected = candidate;
1392 |         break;
1393 |     }
1394 | return {
1395 |     selected: selected,
1396 |     debug: createWeightedDebug(
1397 |         candidates,
1398 |         best,
1399 |         selected,
1400 |         weightedCandidates,
1401 |         windowCp,
1402 |         floorScore,
1403 |         ticketPower,
1404 |         totalTickets,
1405 |     ),
1406 | };
1407 | })(candidates, ctx.settings, principal);
1408 | return { selected: selection.selected, principal: principal, debug: selection.debug };
1409 | }
1410 | function shouldStopAfterStableDepth(result, previousPrincipal, settings) {

```

```

1411 |         return (
1412 |             'hard' === settings.preset &&
1413 |             null !== previousPrincipal &&
1414 |             !! isSameMoveCommand(previousPrincipal.move, result.principal.move) &&
1415 |             !(Math.abs(previousPrincipal.score - result.principal.score) > 90) &&
1416 |             (function rootScoreGap(debug) {
1417 |                 const [bestCandidate, secondCandidate] = debug.candidates;
1418 |                 return bestCandidate && secondCandidate ? bestCandidate.score - secondCandidate.score : 1e7;
1419 |             })(result.debug) ≥ 120
1420 |         );
1421 |     }
1422 |     function negamax(chess, depth, alpha, beta, ply, ctx) {
1423 |         if ((checkTime(ctx), chess.isCheckmate() || chess.isDraw())) return terminalScore(chess, ply);
1424 |         if (depth ≤ 0)
1425 |             return ctx.settings.quiescence
1426 |                 ? quiescence(chess, alpha, beta, ply, ctx)
1427 |                 : evaluatePosition(chess);
1428 |         let bestScore = -1e7,
1429 |             currentAlpha = alpha;
1430 |         const legalMoves = orderMoves(chess.moves({ verbose: !0 }), null);
1431 |         for (const move of legalMoves) {
1432 |             chess.move(toMoveCommand(move));
1433 |             const score = -negamax(chess, depth - 1, -beta, -currentAlpha, ply + 1, ctx);
1434 |             if (
1435 |                 (chess.undo(),
1436 |                 (bestScore = Math.max(bestScore, score)),
1437 |                 (currentAlpha = Math.max(currentAlpha, score)),
1438 |                 currentAlpha ≥ beta)
1439 |             )
1440 |                 break;
1441 |         }
1442 |         return bestScore;
1443 |     }
1444 |     function quiescence(chess, alpha, beta, ply, ctx) {
1445 |         if ((checkTime(ctx), chess.isCheckmate() || chess.isDraw())) return terminalScore(chess, ply);
1446 |         let currentAlpha = alpha;
1447 |         const standPat = evaluatePosition(chess);
1448 |         if (standPat ≥ beta) return beta;
1449 |         if (((currentAlpha = Math.max(currentAlpha, standPat)), ply ≥ 6)) return currentAlpha;
1450 |         const legalMoves = orderMoves(chess.moves({ verbose: !0 }), null),
1451 |             tacticalMoves = chess.isCheck() ? legalMoves : legalMoves.filter(isTacticalMove);
1452 |         for (const move of tacticalMoves) {
1453 |             chess.move(toMoveCommand(move));
1454 |             const score = -quiescence(chess, -beta, -currentAlpha, ply + 1, ctx);
1455 |             if ((chess.undo(), score ≥ beta)) return beta;
1456 |             currentAlpha = Math.max(currentAlpha, score);
1457 |         }
1458 |         return currentAlpha;
1459 |     }
1460 |     function evaluatePosition(chess) {
1461 |         const board = chess.board(),
1462 |             pawns = [],
1463 |             bishops = { w: 0, b: 0 },
1464 |             kings = {};
1465 |         let whiteScore = 0,
1466 |             blackScore = 0,
1467 |             nonPawnMaterial = 0;
1468 |         for (const row of board)
1469 |             for (const pieceOnSquare of row) {
1470 |                 if (null === pieceOnSquare) continue;
1471 |                 const { color: color, type: type, square: square } = pieceOnSquare,
1472 |                     pieceScore = PIECE_VALUES[type] + pieceSquareScore(type, color, square);
1473 |                 ('w' === color ? (whiteScore += pieceScore) : (blackScore += pieceScore),
1474 |                 'p' === type
1475 |                     ? pawns.push({ color: color, file: fileIndex(square), rank: rankNumber(square) })
1476 |                     : ('k' === type && (nonPawnMaterial += PIECE_VALUES[type]),
1477 |                     'b' === type && (bishops[color] += 1),
1478 |                     'k' === type && (kings[color] = square)));
1479 |             }
1480 |         ((whiteScore += bishops.w ≥ 2 ? 35 : 0),
1481 |         (blackScore += bishops.b ≥ 2 ? 35 : 0),

```

```

1482 | | | | | (whiteScore += pawnStructureScore(pawns, 'w')),
1483 | | | | | (blackScore += pawnStructureScore(pawns, 'b')),
1484 | | | | | (whiteScore += openingPrinciplesScore(board, pawns, 'w', nonPawnMaterial)),
1485 | | | | | (blackScore += openingPrinciplesScore(board, pawns, 'b', nonPawnMaterial)),
1486 | | | | | (whiteScore += developmentScore(board, 'w')),
1487 | | | | | (blackScore += developmentScore(board, 'b')),
1488 | | | | | (whiteScore += kingSafetyScore(kings.w, pawns, 'w', nonPawnMaterial)),
1489 | | | | | (blackScore += kingSafetyScore(kings.b, pawns, 'b', nonPawnMaterial)));
1490 | | | | | let score = (whiteScore - blackScore) * ('w' === chess.turn() ? 1 : -1);
1491 | | | | | return ((score += 2 * chess.moves().length), chess.isCheck() && (score -= 45), score);
1492 | | | | | }
1493 | | | | | function pieceSquareScore(type, color, square) {
1494 | | | | |   const file = fileIndex(square),
1495 | | | | |   relativeRank = 'w' === color ? rankNumber(square) : 9 - rankNumber(square),
1496 | | | | |   centerDistance = Math.abs(file - 3.5) + Math.abs(rankNumber(square) - 4.5);
1497 | | | | |   switch (type) {
1498 | | | | |     case 'p':
1499 | | | | |       return 7 * relativeRank + (file ≥ 2 && file ≤ 5 ? 8 : 0);
1500 | | | | |     case 'n':
1501 | | | | |       return 42 - 12 * centerDistance - (1 === relativeRank ? 12 : 0);
1502 | | | | |     case 'b':
1503 | | | | |       return 30 - 7 * centerDistance;
1504 | | | | |     case 'r':
1505 | | | | |       return (relativeRank ≥ 7 ? 18 : 0) + (0 === file || 7 === file ? 4 : 0);
1506 | | | | |     case 'q':
1507 | | | | |       return 14 - 3 * centerDistance;
1508 | | | | |     case 'k':
1509 | | | | |       return relativeRank ≥ 7 ? -18 : 0;
1510 | | | | |   }
1511 | | | | | }
1512 | | | | | function pawnStructureScore(pawns, color) {
1513 | | | | |   const ownPawns = pawns.filter((pawn) => pawn.color === color),
1514 | | | | |   fileCounts = Array.from({ length: 8 }, () => 0);
1515 | | | | |   let score = 0;
1516 | | | | |   for (const pawn of ownPawns) fileCounts[pawn.file] += 1;
1517 | | | | |   for (const pawn of ownPawns) {
1518 | | | | |     const relativeRank = 'w' === color ? pawn.rank : 9 - pawn.rank;
1519 | | | | |     (fileCounts[pawn.file] > 1 && (score -= 14),
1520 | | | | |     0 === (fileCounts[pawn.file - 1] ?? 0) &&
1521 | | | | |     0 === (fileCounts[pawn.file + 1] ?? 0) &&
1522 | | | | |     (score -= 10),
1523 | | | | |     isPassedPawn(pawn, pawns) && (score += (relativeRank - 1) * (relativeRank - 1) * 9));
1524 | | | | |   }
1525 | | | | |   return score;
1526 | | | | | }
1527 | | | | | function isPassedPawn(pawn, pawns) {
1528 | | | | |   const enemyColor = (function oppositeColor(color) {
1529 | | | | |     return 'w' === color ? 'b' : 'w';
1530 | | | | |   })(pawn.color);
1531 | | | | |   return !pawns.some(
1532 | | | | |     (candidate) =>
1533 | | | | |     !(candidate.color === enemyColor || Math.abs(candidate.file - pawn.file) > 1) &&
1534 | | | | |     ('w' === pawn.color ? candidate.rank > pawn.rank : candidate.rank < pawn.rank),
1535 | | | | |   );
1536 | | | | | }
1537 | | | | | function openingPhaseWeight(nonPawnMaterial) {
1538 | | | | |   return clamp(
1539 | | | | |     (nonPawnMaterial - OPENING_PHASE_END_MATERIAL) /
1540 | | | | |     (STARTING_NON_PAWN_MATERIAL - OPENING_PHASE_END_MATERIAL),
1541 | | | | |     0,
1542 | | | | |     1,
1543 | | | | |   );
1544 | | | | | }
1545 | | | | | function openingPrinciplesScore(board, pawns, color, nonPawnMaterial) {
1546 | | | | |   const phase = openingPhaseWeight(nonPawnMaterial);
1547 | | | | |   if (phase ≤ 0) return 0;
1548 | | | | |   let score = (function centralLineOpeningScore(board, color) {
1549 | | | | |     let score = 0;
1550 | | | | |     return (
1551 | | | | |       hasPawnOnSquare(board, color, 'w' === color ? 'd2' : 'd7') || (score += 10),
1552 | | | | |       hasPawnOnSquare(board, color, 'w' === color ? 'e2' : 'e7') || (score += 12),

```

```

1553 |         .score
1554 |     );
1555 |     })(board, color),
1556 |     developedCenterPawns = 0;
1557 |     for (const pawn of pawns) {
1558 |         if (pawn.color !== color) continue;
1559 |         const relativeRank = 'w' === color ? pawn.rank : 9 - pawn.rank;
1560 |         3 === pawn.file || 4 === pawn.file
1561 |             ? (4 === relativeRank || 5 === relativeRank
1562 |                 ? (score += 76)
1563 |                 : 3 === relativeRank && (score += 30),
1564 |             relativeRank ≥ 3 && (developedCenterPawns += 1))
1565 |             : (2 !== pawn.file && 5 !== pawn.file) ||
1566 |             (4 === relativeRank || 5 === relativeRank
1567 |                 ? (score += 18)
1568 |                 : 3 === relativeRank && (score += 6));
1569 |     }
1570 |     return (developedCenterPawns ≥ 2 && (score += 16), Math.round(score * phase));
1571 | }
1572 |
1573 | function hasPawnOnSquare(board, color, square) {
1574 |     for (const row of board)
1575 |         for (const piece of row)
1576 |             if (piece?.color === color && 'p' === piece.type && piece.square === square) return !0;
1577 |     return !1;
1578 | }
1579 |
1580 | function developmentScore(board, color) {
1581 |     const homeRank = 'w' === color ? '1' : '8',
1582 |         knightHomeSquares = 'w' === color ? new Set(['b1', 'g1']) : new Set(['b8', 'g8']),
1583 |         bishopHomeSquares = 'w' === color ? new Set(['c1', 'f1']) : new Set(['c8', 'f8']);
1584 |     let score = 0;
1585 |     for (const row of board)
1586 |         for (const piece of row)
1587 |             null !== piece &&
1588 |             piece.color === color &&
1589 |             ('n' !== piece.type || knightHomeSquares.has(piece.square) || (score += 18),
1590 |             'b' !== piece.type || bishopHomeSquares.has(piece.square) || (score += 16),
1591 |             'q' === piece.type &&
1592 |             piece.square.endsWith(homeRank) &&
1593 |             hasMovedMinorPieces(board, color) &&
1594 |             (score -= 12));
1595 |     return score;
1596 | }
1597 |
1598 | function hasMovedMinorPieces(board, color) {
1599 |     const homeSquares =
1600 |         'w' === color ? new Set(['b1', 'c1', 'f1', 'g1']) : new Set(['b8', 'c8', 'f8', 'g8']);
1601 |     for (const row of board)
1602 |         for (const piece of row)
1603 |             if (
1604 |                 null !== piece &&
1605 |                 piece.color === color &&
1606 |                 ('n' === piece.type || 'b' === piece.type) &&
1607 |                 homeSquares.has(piece.square)
1608 |             )
1609 |                 return !1;
1610 |     return !0;
1611 | }
1612 |
1613 | function kingSafetyScore(kingSquare, pawns, color, nonPawnMaterial) {
1614 |     if (!kingSquare || nonPawnMaterial < 1800) return 0;
1615 |     const kingFile = fileIndex(kingSquare),
1616 |         kingRank = rankNumber(kingSquare),
1617 |         shieldRank = 'w' === color ? kingRank + 1 : kingRank - 1;
1618 |     let score = 2 === kingFile || 6 === kingFile ? 28 : -18;
1619 |     for (let file = kingFile - 1; file ≤ kingFile + 1; file += 1)
1620 |         file < 0 ||
1621 |         file ≥ 8 ||
1622 |         (pawns.some(
1623 |             (pawn) => pawn.color === color && pawn.file === file && pawn.rank === shieldRank,
1624 |         )
1625 |         ? (score += 14)
1626 |         : (score +=
1627 |             openingCentralShieldScore(kingSquare, pawns, color, file, nonPawnMaterial) ?? -10));

```

```

1624 |         return score;
1625 |     }
1626 |     function openingCentralShieldScore(kingSquare, pawns, color, file, nonPawnMaterial) {
1627 |         const homeKingSquare = 'w' === color ? 'e1' : 'e8',
1628 |               phase = openingPhaseWeight(nonPawnMaterial);
1629 |         return kingSquare === homeKingSquare || phase < 0 || (3 === file && 4 === file)
1630 |             ? null
1631 |             : pawns.some((pawn) => {
1632 |                 if (pawn.color !== color || pawn.file !== file) return !1;
1633 |                 const relativeRank = 'w' === color ? pawn.rank : 9 - pawn.rank;
1634 |                 return 3 === relativeRank || 4 === relativeRank;
1635 |             })
1636 |             ? Math.round(24 * phase - 10)
1637 |             : null;
1638 |     }
1639 |     function terminalScore(chess, ply) {
1640 |         return chess.isCheckmate() ? -1e6 + ply : chess.isDraw() ? 0 : evaluatePosition(chess);
1641 |     }
1642 |     function orderMoves(moves, preferredMove) {
1643 |         return [...moves].sort(
1644 |             (left, right) =>
1645 |                 moveOrderingScore(right, preferredMove) - moveOrderingScore(left, preferredMove),
1646 |             );
1647 |     }
1648 |     function moveOrderingScore(move, preferredMove) {
1649 |         let score = 0;
1650 |         return (
1651 |             preferredMove &&
1652 |             (function isSameMove(move, command) {
1653 |                 return (
1654 |                     move.from === command.from &&
1655 |                     move.to === command.to &&
1656 |                     (move.promotion ?? void 0) === (command.promotion ?? void 0)
1657 |                 );
1658 |             })(move, preferredMove) &&
1659 |             (score += 2e6),
1660 |             move.san.includes('#') ? (score += 15e5) : move.san.includes('+') && (score += 16e3),
1661 |             move.isCapture() &&
1662 |             (score +=
1663 |                 12 * (move.captured ? PIECE_VALUES[move.captured] : PIECE_VALUES.p) -
1664 |                 PIECE_VALUES[move.piece]),
1665 |             move.isPromotion() && move.promotion && (score += PIECE_VALUES[move.promotion] + 8e3),
1666 |             (move.isKingsideCastle() || move.isQueensideCastle()) && (score += 1500),
1667 |             (score += 2 * pieceSquareScore(move.piece, move.color, move.to)),
1668 |             score
1669 |         );
1670 |     }
1671 |     function checkTime(ctx) {
1672 |         if (((ctx.nodes += 1), ctx.nodes % 2048 !== 0)) return;
1673 |         const now = Date.now();
1674 |         if (
1675 |             (now - ctx.lastProgressAt >= 100 &&
1676 |             publishSearchProgress(ctx, ctx.progressState, now - ctx.startTime, now),
1677 |             now >= ctx.deadline)
1678 |         )
1679 |             throw new SearchTimeout();
1680 |     }
1681 |     function publishSearchProgress(
1682 |         ctx,
1683 |         progressState,
1684 |         elapsedMs = Date.now() - ctx.startTime,
1685 |         progressAt = Date.now(),
1686 |     ) {
1687 |         ((ctx.progressState = progressState),
1688 |         (ctx.lastProgressAt = progressAt),
1689 |         ctx.onProgress?.({
1690 |             kind: 'progress',
1691 |             id: ctx.requestId,
1692 |             move: progressState.move,
1693 |             depthReached: progressState.depthReached,
1694 |             nodes: ctx.nodes,

```



```

1695 |         .elapsedMs: elapsedMs,
1696 |         .score: progressState.score,
1697 |         .debug: progressState.debug,
1698 |     .}}});
1699 |     .}
1700 |     .function isTacticalMove(move) {
1701 |         .return (
1702 |             .move.isCapture() || move.isPromotion() || move.san.includes('+') || move.san.includes('#')
1703 |         .);
1704 |     .}
1705 |     .function createDeterministicDebug(
1706 |         .candidates,
1707 |         .principal,
1708 |         .selected,
1709 |         .windowCp,
1710 |         .floorScore,
1711 |         .ticketPower,
1712 |     .) {
1713 |         .return {
1714 |             .candidates: sortCandidatesByScore(candidates).map((candidate) => ({
1715 |                 .move: candidate.move,
1716 |                 .san: candidate.san,
1717 |                 .score: candidate.score,
1718 |                 .tickets: isSameMoveCommand(candidate.move, selected.move) ? 1 : 0,
1719 |                 .probability: isSameMoveCommand(candidate.move, selected.move) ? 1 : 0,
1720 |                 .selected: isSameMoveCommand(candidate.move, selected.move),
1721 |                 .principal: isSameMoveCommand(candidate.move, principal.move),
1722 |             .})),
1723 |             .bestScore: principal.score,
1724 |             .windowCp: windowCp,
1725 |             .floorScore: floorScore,
1726 |             .ticketPower: ticketPower,
1727 |             .totalTickets: 1,
1728 |         .};
1729 |     .}
1730 |     .function createWeightedDebug(
1731 |         .candidates,
1732 |         .principal,
1733 |         .selected,
1734 |         .weightedCandidates,
1735 |         .windowCp,
1736 |         .floorScore,
1737 |         .ticketPower,
1738 |         .totalTickets,
1739 |     .) {
1740 |         .return {
1741 |             .candidates: sortCandidatesByScore(candidates).map((candidate) => {
1742 |                 .const weightedCandidate = weightedCandidates.find(({ candidate: weighted }) =>
1743 |                     .isSameMoveCommand(weighted.move, candidate.move),
1744 |                 .),
1745 |                 .tickets = weightedCandidate ? tickets ?? 0;
1746 |             .return {
1747 |                 .move: candidate.move,
1748 |                 .san: candidate.san,
1749 |                 .score: candidate.score,
1750 |                 .tickets: tickets,
1751 |                 .probability: totalTickets > 0 ? tickets / totalTickets : 0,
1752 |                 .selected: isSameMoveCommand(candidate.move, selected.move),
1753 |                 .principal: isSameMoveCommand(candidate.move, principal.move),
1754 |             .});
1755 |         .},
1756 |         .bestScore: principal.score,
1757 |         .windowCp: windowCp,
1758 |         .floorScore: floorScore,
1759 |         .ticketPower: ticketPower,
1760 |         .totalTickets: totalTickets,
1761 |     .};
1762 |     .}
1763 |     .function sortCandidatesByScore(candidates) {
1764 |         .return [ ... candidates ].sort((left, right) => right.score - left.score);
1765 |     .}

```

```

1766 |         function toMoveCommand(move) {
1767 |             return move.promotion
1768 |             ? { from: move.from, to: move.to, promotion: move.promotion }
1769 |             : { from: move.from, to: move.to };
1770 |         }
1771 |         function isSameMoveCommand(left, right) {
1772 |             return (
1773 |                 left.from === right.from &&
1774 |                 left.to === right.to &&
1775 |                 (left.promotion ?? void 0) === (right.promotion ?? void 0)
1776 |             );
1777 |         }
1778 |         function fileIndex(square) {
1779 |             return square.charCodeAt(0) - 97;
1780 |         }
1781 |         function rankNumber(square) {
1782 |             return Number(square[1]);
1783 |         }
1784 |         function clamp(value, min, max) {
1785 |             return Math.min(max, Math.max(min, value));
1786 |         }
1787 |         class SearchTimeout extends Error {}
1788 |         const workerScope = globalThis;
1789 |         workerScope.addEventListener('message', (event) => {
1790 |             const response = (function searchBestMove(request, onProgress) {
1791 |                 const chess = new Chess(request.fen),
1792 |                     settings = (function normalizeAiSettings(settings) {
1793 |                         const timeLimitMs = Math.max(100, Math.round(settings.timeLimitMs));
1794 |                         return {
1795 |                             preset: settings.preset,
1796 |                             maxDepth:
1797 |                                 ((depth = settings.maxDepth),
1798 |                                  (preset = settings.preset),
1799 |                                  'hard' === preset ? HARD_MAX_DEPTH : clamp(Math.round(depth), 1, 5)),
1800 |                             timeLimitMs: timeLimitMs,
1801 |                             randomness: 30,
1802 |                             quiescence: !0,
1803 |                         };
1804 |                     var depth, preset;
1805 |                 })(request.settings),
1806 |                     legalMoves = chess.moves({ verbose: !0 }),
1807 |                     startTime = Date.now(),
1808 |                     ctx = {
1809 |                         requestId: request.id,
1810 |                         settings: settings,
1811 |                         deadline: startTime + settings.timeLimitMs,
1812 |                         startTime: startTime,
1813 |                         nodes: 0,
1814 |                         onProgress: onProgress,
1815 |                         lastProgressAt: startTime,
1816 |                         progressState: { move: null, depthReached: 0, score: 0 },
1817 |                     };
1818 |                 if (0 === legalMoves.length)
1819 |                     return {
1820 |                         id: request.id,
1821 |                         move: null,
1822 |                         depthReached: 0,
1823 |                         nodes: 0,
1824 |                         elapsedMs: 0,
1825 |                         score: terminalScore(chess, 0),
1826 |                     };
1827 |                 let debug,
1828 |                     best = null,
1829 |                     previousBestMove = null,
1830 |                     previousPrincipal = null,
1831 |                     depthReached = 0;
1832 |                 for (let depth = 1; depth ≤ settings.maxDepth; depth += 1) {
1833 |                     try {
1834 |                         const result = searchRoot(chess, depth, ctx, previousBestMove),
1835 |                             elapsedMs = Date.now() - startTime;
1836 |                     if (

```

```

1837 |         ((best = result.selected),
1838 |         (debug = result.debug),
1839 |         (previousBestMove = result.principal.move),
1840 |         (depthReached = depth),
1841 |         publishSearchProgress(
1842 |             ctx,
1843 |             { move: best.move, depthReached: depthReached, score: best.score, debug: debug },
1844 |             elapsedMs,
1845 |             ),
1846 |         shouldStopAfterStableDepth(result, previousPrincipal, settings))
1847 |     )
1848 |     break;
1849 |     previousPrincipal = result.principal;
1850 | } catch (error) {
1851 |     if (error instanceof SearchTimeout) break;
1852 |     throw error;
1853 | }
1854 | if (Date.now() ≥ ctx.deadline) break;
1855 | }
1856 | return (
1857 |     (best ??= (function chooseFallbackMove(chess) {
1858 |         const [move] = orderMoves(chess.moves({ verbose: !0 }), null);
1859 |         if (!move) return { move: { from: 'a1', to: 'a1' }, score: terminalScore(chess, 0) };
1860 |         chess.move(toMoveCommand(move));
1861 |         const score = -evaluatePosition(chess);
1862 |         return (chess.undo(), { move: toMoveCommand(move), score: score });
1863 |     })(chess)),
1864 |     {
1865 |         id: request.id,
1866 |         move: best.move,
1867 |         depthReached: depthReached,
1868 |         nodes: ctx.nodes,
1869 |         elapsedMs: Date.now() - startTime,
1870 |         score: best.score,
1871 |         debug: debug,
1872 |     }
1873 | );
1874 | })(event.data, (progress) => {
1875 |     workerScope.postMessage(progress);
1876 | });
1877 | workerScope.postMessage({ ... response, kind: 'result' });
1878 | });
1879 | })();
1880 | </script>
1881 |
1882 | <script>
1883 |     (() => {
1884 |         const element = document.getElementById('ts-chess-shared-core-source');
1885 |         if (!element) {
1886 |             throw new Error('Missing shared chess runtime source: ts-chess-shared-core-source');
1887 |         }
1888 |         new Function(element.textContent || '')();
1889 |     })();
1890 | </script>
1891 | <script src="./main.js" defer></script>
1892 | <link rel="stylesheet" href="/style.css" />
1893 | </head>
1894 | <body>
1895 |     <main id="chess-app" class="chess-app">
1896 |         <aside class="control-panel" aria-label="게임 컨트롤">
1897 |             <section class="control-group" aria-label="모드">
1898 |                 <span class="control-label">모드</span>
1899 |                 <div class="segmented-control" role="group" aria-label="플레이 모드">
1900 |                     <button class="segmented-button" type="button" data-mode="local">로컬 2인</button>
1901 |                     <button class="segmented-button" type="button" data-mode="ai">AI 상대</button>
1902 |                 </div>
1903 |             </section>
1904 |
1905 |             <section id="color-controls" class="control-group" aria-label="진영">
1906 |                 <span class="control-label">진영</span>
1907 |                 <div class="segmented-control" role="group" aria-label="인가 지역">

```

```

1908 |         <button class="segmented-button" type="button" data-color="w">백</button>
1909 |         <button class="segmented-button" type="button" data-color="b">흑</button>
1910 |     </div>
1911 | </section>
1912 |
1913 |     <section id="ai-controls" class="control-group ai-settings" aria-label="AI 설정">
1914 |         <span class="control-label">AI 설정</span>
1915 |         <div
1916 |             class="segmented-control segmented-control-three"
1917 |             role="group"
1918 |             aria-label="AI 프리셋"
1919 |         >
1920 |             <button class="segmented-button" type="button" data-preset="easy">쉬움</button>
1921 |             <button class="segmented-button" type="button" data-preset="normal">보통</button>
1922 |             <button class="segmented-button" type="button" data-preset="hard">어려움</button>
1923 |         </div>
1924 |         <label class="range-control">
1925 |             <span>최대 깊이<strong id="ai-depth-value"></strong></span>
1926 |             <input id="ai-depth-input" type="range" min="1" max="5" step="1" />
1927 |         </label>
1928 |         <label class="range-control">
1929 |             <span>시간 제한<strong id="ai-time-value"></strong></span>
1930 |             <input id="ai-time-input" type="range" min="500" max="5000" step="250" />
1931 |         </label>
1932 |     </section>
1933 |
1934 |     <section class="control-group clock-settings" aria-label="시간 설정">
1935 |         <span class="control-label">시간</span>
1936 |         <div class="clock-preset-control" role="group" aria-label="시간 프리셋">
1937 |             <button class="segmented-button" type="button" data-clock-preset="1/1">1/1</button>
1938 |             <button class="segmented-button" type="button" data-clock-preset="3/2">3/2</button>
1939 |             <button class="segmented-button" type="button" data-clock-preset="5/3">5/3</button>
1940 |             <button class="segmented-button" type="button" data-clock-preset="10/5">10/5</button>
1941 |             <button class="segmented-button" type="button" data-clock-preset="15/10">15/10</button>
1942 |         </div>
1943 |     </section>
1944 |
1945 |     <section class="action-row" aria-label="게임 동작">
1946 |         <button id="new-game-button" class="primary-button" type="button">새 게임</button>
1947 |         <button id="undo-button" class="ghost-button" type="button">되돌리기</button>
1948 |     </section>
1949 | </aside>
1950 |
1951 | <section class="board-stage" aria-label="체스판">
1952 |     <div id="chess-board" class="chess-board" role="grid" aria-label="체스판"></div>
1953 | </section>
1954 |
1955 | <aside class="clock-panel" aria-label="체스 타이머">
1956 |     <div id="clock-display" class="clock-display">
1957 |         <div id="white-clock" class="clock-card">
1958 |             <span class="clock-side">백</span>
1959 |             <strong id="white-clock-time" class="clock-time"></strong>
1960 |         </div>
1961 |         <div id="black-clock" class="clock-card">
1962 |             <span class="clock-side">흑</span>
1963 |             <strong id="black-clock-time" class="clock-time"></strong>
1964 |         </div>
1965 |     </div>
1966 | </aside>
1967 |
1968 | <aside class="info-panel" aria-label="게임 정보">
1969 |     <section class="status-panel" aria-live="polite">
1970 |         <span class="control-label">상태</span>
1971 |         <strong id="game-status" class="game-status"></strong>
1972 |         <dl class="status-grid">
1973 |             <div>
1974 |                 <dt>차례</dt>
1975 |                 <dd id="turn-meta"></dd>
1976 |             </div>
1977 |             <div>
1978 |                 <dt>모드</dt>

```

```

1979 | . . . . . <dd id="mode-meta"></dd>
1980 | . . . . . </div>
1981 | . . . . . <div>
1982 | . . . . . <dt>수순</dt>
1983 | . . . . . <dd id="move-meta"></dd>
1984 | . . . . . </div>
1985 | . . . . . </dl>
1986 | . . . . . </section>
1987 |
1988 | . . . . . <section id="ai-candidate-panel" class="ai-candidate-panel" aria-label="AI 정보" hidden>
1989 | . . . . . <div class="ai-candidate-header">
1990 | . . . . . <span class="control-label">AI 정보</span>
1991 | . . . . . <span id="ai-candidate-meta" class="ai-candidate-meta"></span>
1992 | . . . . . </div>
1993 | . . . . . <dl class="ai-info-grid">
1994 | . . . . . <div>
1995 | . . . . . <dt>상태</dt>
1996 | . . . . . <dd id="ai-info-status"></dd>
1997 | . . . . . </div>
1998 | . . . . . <div>
1999 | . . . . . <dt>난이도</dt>
2000 | . . . . . <dd id="ai-info-preset"></dd>
2001 | . . . . . </div>
2002 | . . . . . <div>
2003 | . . . . . <dt>인간</dt>
2004 | . . . . . <dd id="ai-info-human"></dd>
2005 | . . . . . </div>
2006 | . . . . . <div>
2007 | . . . . . <dt>depth</dt>
2008 | . . . . . <dd id="ai-info-depth"></dd>
2009 | . . . . . </div>
2010 | . . . . . <div>
2011 | . . . . . <dt>nodes</dt>
2012 | . . . . . <dd id="ai-info-nodes"></dd>
2013 | . . . . . </div>
2014 | . . . . . <div>
2015 | . . . . . <dt>score</dt>
2016 | . . . . . <dd id="ai-info-score"></dd>
2017 | . . . . . </div>
2018 | . . . . . </dl>
2019 | . . . . . <div class="ai-candidate-table" role="table" aria-label="AI 후보 수 평가">
2020 | . . . . . <div class="ai-candidate-row ai-candidate-head" role="row">
2021 | . . . . . <span role="columnheader">SAN</span>
2022 | . . . . . <span role="columnheader">cp</span>
2023 | . . . . . <span role="columnheader">weight</span>
2024 | . . . . . <span role="columnheader">prob</span>
2025 | . . . . . </div>
2026 | . . . . . <div id="ai-candidate-list" class="ai-candidate-list"></div>
2027 | . . . . . </div>
2028 | . . . . . </section>
2029 |
2030 | . . . . . <section class="history-panel" aria-label="기보">
2031 | . . . . . <div class="history-header">
2032 | . . . . . <span class="control-label">기보</span>
2033 | . . . . . </div>
2034 | . . . . . <div id="move-list" class="move-list"></div>
2035 | . . . . . </section>
2036 | . . . . . </aside>
2037 |
2038 | . . . . . <div id="promotion-overlay" class="promotion-overlay" hidden>
2039 | . . . . . <div
2040 | . . . . . class="promotion-dialog"
2041 | . . . . . role="dialog"
2042 | . . . . . aria-modal="true"
2043 | . . . . . aria-labelledby="promotion-title"
2044 | . . . . . >
2045 | . . . . . <h2 id="promotion-title">승격</h2>
2046 | . . . . . <div
2047 | . . . . . id="promotion-choices"
2048 | . . . . . class="promotion-choices"
2049 | . . . . . role="group"

```

```

2050 | | . | . | . | . | . | . | . | aria-label="승격·기물"
2051 | | . | . | . | . | . | . | . | </div>
2052 | | . | . | . | . | . | . | . | </div>
2053 | | . | . | . | . | . | . | . | </div>
2054 | | . | . | . | . | . | . | . | </main>
2055 | | . | . | . | . | . | . | . | </body>
2056 | | . | . | . | . | . | . | . | </html>
2057 |

```

```
==== main.js (33823 B / 04a8e344941e) ====
```

```

1 | const sharedCore = globalThis.TSChessShared;
2 | if (!sharedCore) throw new Error('Missing shared chess runtime: TSChessShared');
3 | const Chess = sharedCore.Chess,
4 |   SQUARES = sharedCore.SQUARES,
5 |   HARD_MAX_DEPTH = Number.POSITIVE_INFINITY,
6 |   AI_PRESETS = {
7 |     easy: { preset: 'easy', maxDepth: 2, timeLimitMs: 750, randomness: 30, quiescence: !0 },
8 |     normal: { preset: 'normal', maxDepth: 5, timeLimitMs: 5e3, randomness: 30, quiescence: !0 },
9 |     hard: {
10 |       preset: 'hard',
11 |       maxDepth: HARD_MAX_DEPTH,
12 |       timeLimitMs: 5e3,
13 |       randomness: 30,
14 |       quiescence: !0,
15 |     },
16 |   },
17 |   DEFAULT_AI_SETTINGS = { ...AI_PRESETS.normal };
18 | function normalizeAiSettings(settings) {
19 |   const timeLimitMs = Math.max(100, Math.round(settings.timeLimitMs));
20 |   return {
21 |     preset: settings.preset,
22 |     maxDepth:
23 |       ((depth = settings.maxDepth),
24 |       (preset = settings.preset),
25 |       'hard' === preset
26 |       ? HARD_MAX_DEPTH
27 |       : (function clamp(value, min, max) {
28 |         return Math.min(max, Math.max(min, value));
29 |       })(Math.round(depth), 1, 5)),
30 |     timeLimitMs: timeLimitMs,
31 |     randomness: 30,
32 |     quiescence: !0,
33 |   };
34 |   var depth, preset;
35 | }
36 | const jsContent = (() => {
37 |   const sharedElement = document.getElementById('ts-chess-shared-core-source');
38 |   const workerElement = document.getElementById('ai-worker-source-1');
39 |   if (!sharedElement) {
40 |     throw new Error('Missing shared chess runtime source: ts-chess-shared-core-source');
41 |   }
42 |   if (!workerElement) {
43 |     throw new Error('Missing inline worker source: ai-worker-source-1');
44 |   }
45 |   return (sharedElement.textContent || '') + '\n' + (workerElement.textContent || '');
46 | })(),
47 | blob =
48 |   'undefined' !== typeof self &&
49 |   self.Blob &&
50 |   new Blob([(self.URL || self.webkitURL).revokeObjectURL(self.location.href);', jsContent], {
51 |     type: 'text/javascript; charset=utf-8',
52 |   });
53 | function WorkerWrapper(options) {
54 |   let objURL;
55 |   try {
56 |     if (((objURL = blob && (self.URL || self.webkitURL).createObjectURL(blob)), !objURL)) throw '';
57 |     const worker = new Worker(objURL, { name: options?.name });
58 |     return (
59 |       worker.addEventListener('error', () => {
60 |         (self.URL || self.webkitURL).revokeObjectURL(objURL);
61 |       })

```

```

62 |     worker
63 |   );
64 | } catch (e) {
65 |   return new Worker('data:text/javascript;charset=utf-8,' + encodeURIComponent(jsContent), {
66 |     name: options?.name,
67 |   });
68 | }
69 | }
70 | const BOARD_FILES = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'],
71 |   BLACK_FILES = [...BOARD_FILES].reverse(),
72 |   WHITE_RANKS = [8, 7, 6, 5, 4, 3, 2, 1],
73 |   BLACK_RANKS = [...WHITE_RANKS].reverse(),
74 |   PROMOTION_PIECES = ['q', 'r', 'b', 'n'],
75 |   SQUARE_SET = new Set(SQUARES),
76 |   AI_WHITE_OPENING_PRESET = [
77 |     { from: 'e2', to: 'e4', weight: 28 },
78 |     { from: 'd2', to: 'd4', weight: 25 },
79 |     { from: 'g1', to: 'f3', weight: 24 },
80 |     { from: 'c2', to: 'c4', weight: 20 },
81 |     { from: 'g2', to: 'g3', weight: 9 },
82 |     { from: 'b2', to: 'b3', weight: 6 },
83 |   ],
84 |   CLOCK_PRESETS = {
85 |     '1/1': { baseMs: 6e4, incrementMs: 1e3 },
86 |     '3/2': { baseMs: 18e4, incrementMs: 2e3 },
87 |     '5/3': { baseMs: 3e5, incrementMs: 3e3 },
88 |     '10/5': { baseMs: 6e5, incrementMs: 5e3 },
89 |     '15/10': { baseMs: 9e5, incrementMs: 1e4 },
90 |   },
91 |   PIECE_GLYPHS = {
92 |     w: { p: '♔', n: '♖', b: '♜', r: '♞', q: '♝', k: '♚' },
93 |     b: { p: '♚', n: '♞', b: '♜', r: '♖', q: '♝', k: '♔' },
94 |   };
95 | function requireElement(selector, root) {
96 |   const element = root.querySelector(selector);
97 |   if (!element) throw new Error(`Missing element: ${selector}`);
98 |   return element;
99 | }
100 | function createCoordinate(type, label) {
101 |   const coordinate = document.createElement('span');
102 |   return (
103 |     (coordinate.className = `coordinate coordinate-${type}`),
104 |     (coordinate.textContent = label),
105 |     coordinate
106 |   );
107 | }
108 | function toSquare(file, rank) {
109 |   return `${file}${rank}`;
110 | }
111 | function createClockState(preset) {
112 |   const baseMs = CLOCK_PRESETS[preset].baseMs;
113 |   return { w: baseMs, b: baseMs };
114 | }
115 | function colorName(color) {
116 |   return 'w' === color ? '백' : '흑';
117 | }
118 | function presetName(preset) {
119 |   switch (preset) {
120 |     case 'easy':
121 |       return '쉬움';
122 |     case 'normal':
123 |       return '보통';
124 |     case 'hard':
125 |       return '어려움';
126 |   }
127 | }
128 | function formatSeconds(milliseconds) {
129 |   return `${(milliseconds / 1e3).toFixed(milliseconds % 1e3 === 0 ? 0 : 2)}초`;
130 | }
131 | function formatDebugScore(score) {
132 |   return Math.abs(score) ≥ 9e5

```

```

133 |     .? .score > 0
134 |     .? .'+mate'
135 |     .: .'-mate'
136 |     .: .score > 0
137 |     .? .`+${Math.round(score)}`
138 |     .: .String(Math.round(score));
139 | }
140 | function oppositeColor(color) {
141 |     return 'w' === color ? 'b' : 'w';
142 | }
143 | function getSquareLabel(square, piece) {
144 |     return piece ? `${square}-${colorName(piece.color)}-${pieceName(piece.type)}` : `${square} 빈 칸`;
145 | }
146 | function pieceName(piece) {
147 |     switch (piece) {
148 |     case 'p':
149 |         return '폰';
150 |     case 'n':
151 |         return '나이트';
152 |     case 'b':
153 |         return '비숍';
154 |     case 'r':
155 |         return '룩';
156 |     case 'q':
157 |         return '퀸';
158 |     case 'k':
159 |         return '킹';
160 |     }
161 | }
162 | !(function createChessApp() {
163 |     const root = requireElement('#chess-app', document),
164 |           boardElement = requireElement('#chess-board', root),
165 |           colorControlsElement = requireElement('#color-controls', root),
166 |           aiControlsElement = requireElement('#ai-controls', root),
167 |           statusElement = requireElement('#game-status', root),
168 |           turnMetaElement = requireElement('#turn-meta', root),
169 |           modeMetaElement = requireElement('#mode-meta', root),
170 |           moveMetaElement = requireElement('#move-meta', root),
171 |           aiCandidatePanelElement = requireElement('#ai-candidate-panel', root),
172 |           aiCandidateMetaElement = requireElement('#ai-candidate-meta', root),
173 |           aiInfoStatusElement = requireElement('#ai-info-status', root),
174 |           aiInfoPresetElement = requireElement('#ai-info-preset', root),
175 |           aiInfoHumanElement = requireElement('#ai-info-human', root),
176 |           aiInfoDepthElement = requireElement('#ai-info-depth', root),
177 |           aiInfoNodesElement = requireElement('#ai-info-nodes', root),
178 |           aiInfoScoreElement = requireElement('#ai-info-score', root),
179 |           aiCandidateListElement = requireElement('#ai-candidate-list', root),
180 |           moveListElement = requireElement('#move-list', root),
181 |           clockDisplayElement = requireElement('#clock-display', root),
182 |           whiteClockElement = requireElement('#white-clock', root),
183 |           blackClockElement = requireElement('#black-clock', root),
184 |           whiteClockTimeElement = requireElement('#white-clock-time', root),
185 |           blackClockTimeElement = requireElement('#black-clock-time', root),
186 |           newGameButton = requireElement('#new-game-button', root),
187 |           undoButton = requireElement('#undo-button', root),
188 |           aiDepthInput = requireElement('#ai-depth-input', root),
189 |           aiDepthValue = requireElement('#ai-depth-value', root),
190 |           aiTimeInput = requireElement('#ai-time-input', root),
191 |           aiTimeValue = requireElement('#ai-time-value', root),
192 |           promotionOverlay = requireElement('#promotion-overlay', root),
193 |           promotionChoices = requireElement('#promotion-choices', root),
194 |           modeButtons = Array.from(root.querySelectorAll('[data-mode]')),
195 |           colorButtons = Array.from(root.querySelectorAll('[data-color]')),
196 |           presetButtons = Array.from(root.querySelectorAll('[data-preset]')),
197 |           clockPresetButtons = Array.from(root.querySelectorAll('[data-clock-preset]')),
198 |           game = new Chess();
199 |     let mode = 'local',
200 |         clockPreset = '10/5',
201 |         clockMs = createClockState(clockPreset),
202 |         activeClockColor = null,
203 |         lastClockTick = Date.now(),

```



```

204 |     .clockTimerId = null,
205 |     .timedOutColor = null,
206 |     .humanColor = 'w',
207 |     .selectedSquare = null,
208 |     .legalMoves = [],
209 |     .pendingPromotion = null,
210 |     .aiThinking = !1,
211 |     .aiTimerId = null,
212 |     .aiWorker = null,
213 |     .aiRequestId = 0,
214 |     .activeAiRequestId = null,
215 |     .activeAiFen = null,
216 |     .aiSettings = { ... DEFAULT_AI_SETTINGS },
217 |     .aiProgress = null;
218 |     function selectSquare(square) {
219 |       ((selectedSquare = square),
220 |       (legalMoves = game.moves({ verbose: !0, square: square })),
221 |       render());
222 |     }
223 |     function playMove(moveCommand) {
224 |       if (!commitMove(moveCommand)) return (clearSelection(), void render());
225 |       ((pendingPromotion = null), clearSelection(), render(), scheduleAiIfNeeded());
226 |     }
227 |     function resetGame() {
228 |       (cancelAiMove(),
229 |       game.reset(),
230 |       (pendingPromotion = null),
231 |       clearSelection(),
232 |       resetClockState(),
233 |       restartClockForCurrentTurn(),
234 |       render(),
235 |       scheduleAiIfNeeded());
236 |     }
237 |     function scheduleAiIfNeeded() {
238 |       'ai' !== mode ||
239 |       game.isGameOver() ||
240 |       null !== timedOutColor ||
241 |       game.turn() === humanColor ||
242 |       aiThinking ||
243 |       pendingPromotion ||
244 |       (function tryCommitAiWhiteOpeningPreset() {
245 |         if ('b' !== humanColor || 'w' !== game.turn() || game.history().length > 0) return !1;
246 |         const selectedMove = (function pickWeightedOpeningMove(moves) {
247 |           const totalWeight = moves.reduce((total, move) => total + move.weight, 0);
248 |           if (totalWeight <= 0) return null;
249 |           let draw = Math.random() * totalWeight;
250 |           for (const move of moves)
251 |             if (((draw -= move.weight), draw <= 0))
252 |               return { from: move.from, to: move.to, promotion: move.promotion };
253 |           const fallback = moves.at(-1);
254 |           return fallback
255 |             ? { from: fallback.from, to: fallback.to, promotion: fallback.promotion }
256 |             : null;
257 |         }))(
258 |           AI_WHITE_OPENING_PRESET.filter((presetMove) =>
259 |             game.moves({ verbose: !0, square: presetMove.from }).some((move) =>
260 |               (function isSameMoveCommand(move, command) {
261 |                 return (
262 |                   move.from === command.from &&
263 |                   move.to === command.to &&
264 |                   (move.promotion ?? void 0) === (command.promotion ?? void 0)
265 |                 );
266 |               })(move, presetMove),
267 |             ),
268 |           ),
269 |         );
270 |     return !(
271 |       !selectedMove ||
272 |       !commitMove(selectedMove) ||
273 |       ((aiProgress = null), clearSelection(), render(), 0)
274 |     );

```

```

275 |         })();
276 |         ((aiThinking !== 0),
277 |         (aiProgress === null),
278 |         render(),
279 |         (aiTimerId = window.setTimeout(() => {
280 |             ((aiTimerId === null),
281 |             (function startAiSearch() {
282 |                 aiWorker?.terminate();
283 |                 const requestId = aiRequestId + 1,
284 |                 fen = game.fen();
285 |                 ((aiRequestId = requestId),
286 |                 (activeAiRequestId = requestId),
287 |                 (activeAiFen = fen),
288 |                 (aiWorker = new WorkerWrapper()),
289 |                 aiWorker.addEventListener('message', (event) => {
290 |                     !(function handleAiMessage(message) {
291 |                         if (
292 |                             message.id === activeAiRequestId &&
293 |                             null !== activeAiFen &&
294 |                             game.fen() === activeAiFen
295 |                         ) {
296 |                             if (((aiProgress = message), 'progress' === message.kind))
297 |                                 return (renderStatus(), void renderAiCandidateDebug());
298 |                             ((aiThinking !== 1),
299 |                             (activeAiRequestId = null),
300 |                             (activeAiFen = null),
301 |                             aiWorker?.terminate(),
302 |                             (aiWorker = null),
303 |                             message.move && commitMove(message.move),
304 |                             clearSelection(),
305 |                             render());
306 |                         }
307 |                     })(event.data);
308 |                 })),
309 |                 aiWorker.addEventListener('error', () => {
310 |                     activeAiRequestId === requestId &&
311 |                     ((aiThinking !== 1),
312 |                     (activeAiRequestId = null),
313 |                     (activeAiFen = null),
314 |                     aiWorker?.terminate(),
315 |                     (aiWorker = null),
316 |                     render());
317 |                 })),
318 |                 aiWorker.postMessage({
319 |                     id: requestId,
320 |                     fen: fen,
321 |                     settings: resolveAiSettingsForSearch(),
322 |                 }));
323 |             })();
324 |         }, 180));
325 |     }
326 |     function cancelAiMove() {
327 |         ((aiRequestId += 1),
328 |         (activeAiRequestId = null),
329 |         (activeAiFen = null),
330 |         (aiProgress = null),
331 |         null !== aiTimerId && (window.clearTimeout(aiTimerId), (aiTimerId = null)),
332 |         aiWorker && (aiWorker.terminate(), (aiWorker = null)),
333 |         (aiThinking !== 1));
334 |     }
335 |     function commitMove(moveCommand) {
336 |         const movingColor = game.turn();
337 |         if (!flushClockElapsed()) return !1;
338 |         try {
339 |             game.move(moveCommand);
340 |         } catch {
341 |             return !1;
342 |         }
343 |         return (
344 |             (clockMs[movingColor] += CLOCK_PRESETS[clockPreset].incrementMs),
345 |             restartClockForCurrentTurn(),

```

```

346 |         ····!0
347 |     ····);
348 | }
349 | function updateAiSettings(settings)·{
350 |     ····((aiSettings·=·normalizeAiSettings(settings)),
351 |     ····aiThinking·&&·cancelAiMove(),
352 |     ····render(),
353 |     ····scheduleAiIfNeeded());
354 | }
355 | function resolveAiSettingsForSearch()·{
356 |     ····const settings·=·normalizeAiSettings(aiSettings);
357 |     ····if·('hard'·≠·settings.preset)·return settings;
358 |     ····const aiColor·=·game.turn(),
359 |     ····remainingMs·=·clockMs[aiColor],
360 |     ····targetMs·=·remainingMs·/·30·+·0.8·*·CLOCK_PRESETS[clockPreset].incrementMs,
361 |     ····timeLimitMs·=·remainingMs·≤·3e3·?·Math.min(targetMs,·0.35·*·remainingMs)·:·targetMs;
362 |     ····return normalizeAiSettings({·... settings,·timeLimitMs:·timeLimitMs});
363 | }
364 | function resetClockState()·{
365 |     ····((clockMs·=·createClockState(clockPreset)),
366 |     ····(timedOutColor·=·null),
367 |     ····(activeClockColor·=·null),
368 |     ····(lastClockTick·=·Date.now()));
369 | }
370 | function restartClockForCurrentTurn()·{
371 |     ····if·((clearClockTimer(),·game.isGameOver()·||·null·≠·timedOutColor))
372 |     ····return·((activeClockColor·=·null),·void·renderClocks());
373 |     ····((activeClockColor·=·game.turn()),
374 |     ····(lastClockTick·=·Date.now()),
375 |     ····(clockTimerId·=·window.setInterval(tickClock,·100)),
376 |     ····renderClocks());
377 | }
378 | function clearClockTimer()·{
379 |     ····null·≠·clockTimerId·&&·(window.clearInterval(clockTimerId),·(clockTimerId·=·null));
380 | }
381 | function tickClock()·{
382 |     ····flushClockElapsed()·&&·renderClocks();
383 | }
384 | function flushClockElapsed()·{
385 |     ····if·(null·===·activeClockColor·||·null·≠·timedOutColor·||·game.isGameOver())·return·!0;
386 |     ····const now·=·Date.now(),
387 |     ····elapsedMs·=·Math.max(0,·now·-·lastClockTick);
388 |     ····return·(
389 |     ····(lastClockTick·=·now),
390 |     ····0·===·elapsedMs·||
391 |     ····((clockMs[activeClockColor]·=·Math.max(0,·clockMs[activeClockColor]·-·elapsedMs)),
392 |     ····0·≠·clockMs[activeClockColor]·||
393 |     ····((function·handleClockTimeout(color)·{
394 |     ········((timedOutColor·=·color),
395 |     ········(activeClockColor·=·null),
396 |     ········(clockMs[color]·=·0),
397 |     ········clearClockTimer(),
398 |     ········cancelAiMove(),
399 |     ········clearSelection(),
400 |     ········render());
401 |     ·····})·(activeClockColor),
402 |     ····!1))
403 |     ····);
404 | }
405 | function clearSelection()·{
406 |     ····((selectedSquare·=·null),·(legalMoves·=·[]));
407 | }
408 | function render()·{
409 |     ····(!function·renderControls()·{
410 |     ········(modeButtons.forEach((button)·=>·{
411 |     ········const isActive·=·button.dataset.mode·===·mode;
412 |     ········(button.classList.toggle('is-active',·isActive),
413 |     ········button.setAttribute('aria-pressed',·String(isActive)));
414 |     ········}),
415 |     ····colorButtons.forEach((button)·=>·{
416 |     ········const isActive·=·button.dataset.color·===·humanColor;

```

```

417 |         (button.classList.toggle('is-active', isActive),
418 |         button.setAttribute('aria-pressed', String(isActive)));
419 |     },
420 |     (colorControlsElement.hidden = 'ai' !== mode),
421 |     (aiControlsElement.hidden = 'ai' !== mode),
422 |     (function renderAiSettingsControls() {
423 |         (presetButtons.forEach((button) => {
424 |             const isActive = button.dataset.preset === aiSettings.preset;
425 |             (button.classList.toggle('is-active', isActive),
426 |             button.setAttribute('aria-pressed', String(isActive)));
427 |         })),
428 |         (aiDepthInput.value =
429 |         'hard' === aiSettings.preset ? aiDepthInput.max : String(aiSettings.maxDepth)),
430 |         (aiDepthInput.disabled = 'hard' === aiSettings.preset),
431 |         (aiDepthValue.textContent =
432 |         'hard' === aiSettings.preset ? '무제한' : `${aiSettings.maxDepth}`),
433 |         (aiTimeInput.value = String(aiSettings.timeLimitMs)),
434 |         (aiTimeInput.disabled = 'hard' === aiSettings.preset),
435 |         (aiTimeValue.textContent =
436 |         'hard' === aiSettings.preset ? '자동' : `${formatSeconds(aiSettings.timeLimitMs)}`));
437 |     })(),
438 |     (function renderClockSettingsControls() {
439 |         clockPresetButtons.forEach((button) => {
440 |             const isActive = button.dataset.clockPreset === clockPreset;
441 |             (button.classList.toggle('is-active', isActive),
442 |             button.setAttribute('aria-pressed', String(isActive)));
443 |         });
444 |     })(),
445 |     (undoButton.disabled = 0 === game.history().length));
446 | })(),
447 | renderClocks(),
448 | (function renderBoard() {
449 |     const fragment = document.createDocumentFragment(),
450 |     orientation = getBoardOrientation(),
451 |     files = 'w' === orientation ? BOARD_FILES : BLACK_FILES,
452 |     ranks = 'w' === orientation ? WHITE_RANKS : BLACK_RANKS,
453 |     lastMove = (function getLastMove() {
454 |         return game.history({ verbose: !0 }).at(-1) ?? null;
455 |     })();
456 |     boardElement.classList.toggle('is-locked', !canUseBoard());
457 |     for (const rank of ranks)
458 |         for (const file of files) {
459 |             const square = toSquare(file, rank),
460 |             piece = game.get(square),
461 |             button = document.createElement('button');
462 |             if (
463 |                 ((button.type = 'button'),
464 |                 (button.className = getSquareClasses(square, lastMove)),
465 |                 (button.dataset.square = square),
466 |                 (button.disabled = !canUseBoard()),
467 |                 button.setAttribute('role', 'gridcell'),
468 |                 button.setAttribute('aria-label', getSquareLabel(square, piece)),
469 |                 file === files[0] && button.appendChild(createCoordinate('rank', String(rank))),
470 |                 rank === ranks[ranks.length - 1] &&
471 |                 button.appendChild(createCoordinate('file', file)),
472 |                 piece)
473 |             ) {
474 |                 const pieceElement = document.createElement('span');
475 |                 ((pieceElement.className = 'piece'),
476 |                 (pieceElement.textContent = PIECE_GLYPHS[piece.color][piece.type]),
477 |                 button.appendChild(pieceElement));
478 |             }
479 |             fragment.appendChild(button);
480 |         }
481 |     boardElement.replaceChildren(fragment);
482 | })(),
483 | renderStatus(),
484 | renderAiCandidateDebug(),
485 | (function renderMoveList() {
486 |     const history = game.history({ verbose: !0 }),
487 |     fragment = document.createDocumentFragment();

```

```

488 |         if (0 === history.length) {
489 |             const empty = document.createElement('p');
490 |             return (
491 |                 (empty.className = 'move-empty'),
492 |                 (empty.textContent = '아직 둔 수가 없습니다.'),
493 |                 fragment.appendChild(empty),
494 |                 void moveListElement.replaceChildren(fragment)
495 |             );
496 |         }
497 |         for (let index = 0; index < history.length; index += 2) {
498 |             const row = document.createElement('div'),
499 |                 moveNumber = document.createElement('span'),
500 |                 whiteMove = document.createElement('span'),
501 |                 blackMove = document.createElement('span');
502 |             (row.className = 'move-row'),
503 |             (moveNumber.className = 'move-number'),
504 |             (whiteMove.className = 'move-san'),
505 |             (blackMove.className = 'move-san'),
506 |             (moveNumber.textContent = `${Math.floor(index / 2) + 1}.`),
507 |             (whiteMove.textContent = history[index] ? .san ?? ''),
508 |             (blackMove.textContent = history[index + 1] ? .san ?? ''),
509 |             row.append(moveNumber, whiteMove, blackMove),
510 |             fragment.appendChild(row);
511 |         }
512 |         (moveListElement.replaceChildren(fragment),
513 |         (moveListElement.scrollTop = moveListElement.scrollHeight));
514 |     })(),
515 |     (function renderPromotionDialog() {
516 |         if (
517 |             ((promotionOverlay.hidden = null === pendingPromotion),
518 |             promotionChoices.replaceChildren(),
519 |             pendingPromotion)
520 |         )
521 |             for (const piece of PROMOTION_PIECES) {
522 |                 const button = document.createElement('button');
523 |                 ((button.className = 'promotion-button'),
524 |                 (button.type = 'button'),
525 |                 (button.dataset.promotion = piece),
526 |                 button.setAttribute('aria-label', `${pieceName(piece)} 승격`),
527 |                 (button.textContent = PIECE_GLYPHS[game.turn()][piece]),
528 |                 promotionChoices.appendChild(button));
529 |             }
530 |         })();
531 |     }
532 |     function renderClocks() {
533 |         const bottomColor = getBoardOrientation(),
534 |             topClockElement = 'w' === oppositeColor(bottomColor) ? whiteClockElement : blackClockElement,
535 |             bottomClockElement = 'w' === bottomColor ? whiteClockElement : blackClockElement;
536 |         ((clockDisplayElement.children[0] === topClockElement &&
537 |         clockDisplayElement.children[1] === bottomClockElement) ||
538 |         clockDisplayElement.replaceChildren(topClockElement, bottomClockElement),
539 |         renderClockPosition('w', bottomColor),
540 |         renderClockPosition('b', bottomColor),
541 |         renderClock('w', whiteClockElement, whiteClockTimeElement),
542 |         renderClock('b', blackClockElement, blackClockTimeElement));
543 |     }
544 |     function renderClockPosition(color, bottomColor) {
545 |         const clockElement = 'w' === color ? whiteClockElement : blackClockElement,
546 |             isBottom = color === bottomColor;
547 |         (clockElement.classList.toggle('is-bottom', isBottom),
548 |         clockElement.classList.toggle('is-top', !isBottom));
549 |     }
550 |     function renderClock(color, clockElement, timeElement) {
551 |         const remainingMs = clockMs[color],
552 |             isActive = activeClockColor === color && null === timedOutColor,
553 |             isTimedOut = timedOutColor === color;
554 |         (clockElement.classList.toggle('is-active', isActive),
555 |         clockElement.classList.toggle('is-warning', remainingMs ≤ 3e4),
556 |         clockElement.classList.toggle('is-danger', remainingMs ≤ 1e4),
557 |         clockElement.classList.toggle('is-timeout', isTimedOut),
558 |         (timeElement.textContent = (function formatClockTime(milliseconds) {

```

```

559 |         const safeMilliseconds = Math.max(0, milliseconds);
560 |         if (safeMilliseconds < 1e4)
561 |             return `0:${(safeMilliseconds / 1e3).toFixed(1).padStart(4, '0')}`;
562 |         const totalSeconds = Math.ceil(safeMilliseconds / 1e3);
563 |         return `${Math.floor(totalSeconds / 60)}:${String(totalSeconds % 60).padStart(2, '0')}`;
564 |     })(remainingMs));
565 | }
566 | function renderStatus() {
567 |     const historyLength = game.history().length;
568 |     ((statusElement.textContent = (function getStatusText() {
569 |         return game.isCheckmate()
570 |             ? `${colorName(oppositeColor(game.turn()))} 체크메이트 승리`
571 |             : game.isStalemate()
572 |                 ? '스테일메이트 무승부'
573 |                 : game.isInsufficientMaterial()
574 |                     ? '기물 부족 무승부'
575 |                     : game.isThreefoldRepetition()
576 |                         ? '3회 반복 무승부'
577 |                         : game.isDrawByFiftyMoves()
578 |                             ? '50수 규칙 무승부'
579 |                             : game.isDraw()
580 |                                 ? '무승부'
581 |                                 : timedOutColor
582 |                                     ? [
583 |                                         `${colorName(timedOutColor)} 시간 초과`,
584 |                                         `${colorName(oppositeColor(timedOutColor))} 승리`,
585 |                                     ].join('...')
586 |                                     : aiThinking
587 |                                         ? 'AI 계산 중'
588 |                                         : game.isCheck()
589 |                                             ? `${colorName(game.turn())} 체크`
590 |                                             : `${colorName(game.turn())} 차례`;
591 |     })()),
592 |     (turnMetaElement.textContent = colorName(game.turn())),
593 |     (modeMetaElement.textContent =
594 |         'local' === mode
595 |         ? '로컬 2인'
596 |         : ['AI 상대', `인간 ${colorName(humanColor)}`, presetName(aiSettings.preset)].join(
597 |             '...',
598 |             ')),
599 |     (moveMetaElement.textContent = [` ${game.moveNumber()}수`, `${historyLength} half-move`].join(
600 |         '...',
601 |         ')));
602 | }
603 | function renderAiCandidateDebug() {
604 |     if ('ai' !== mode)
605 |         return (
606 |             (aiCandidatePanelElement.hidden = !0),
607 |             (aiCandidateMetaElement.textContent = ''),
608 |             (aiInfoStatusElement.textContent = ''),
609 |             (aiInfoPresetElement.textContent = ''),
610 |             (aiInfoHumanElement.textContent = ''),
611 |             (aiInfoDepthElement.textContent = ''),
612 |             (aiInfoNodesElement.textContent = ''),
613 |             (aiInfoScoreElement.textContent = ''),
614 |             void aiCandidateListElement.replaceChildren()
615 |         );
616 |     const debug = aiProgress?.debug;
617 |     if (
618 |         ((aiCandidatePanelElement.hidden = !1),
619 |         (aiCandidateMetaElement.textContent = aiProgress
620 |             ? `경과 ${formatSeconds(aiProgress.elapsedMs)}`
621 |             : '')),
622 |         (aiInfoStatusElement.textContent = (function getAiInfoStatusText() {
623 |             return game.isGameOver() || null !== timedOutColor
624 |                 ? '대국 종료'
625 |                 : aiThinking
626 |                     ? aiProgress
627 |                     ? '계산 중'
628 |                     : '계산 준비 중'
629 |                     : game.turn() === humanColor

```

```

630 |         ? '대기·중'
631 |         : '응수·대기';
632 |     })),
633 |     (aiInfoPresetElement.textContent = presetName(aiSettings.preset)),
634 |     (aiInfoHumanElement.textContent = colorName(humanColor)),
635 |     (aiInfoDepthElement.textContent = aiProgress ? String(aiProgress.depthReached) : '-'),
636 |     (aiInfoNodesElement.textContent = aiProgress
637 |     ? `${(function formatNodes(nodes) {
638 |         return nodes ≥ 1e6 ? `${Math.round(nodes / 1e3)}k` : String(nodes);
639 |     })(aiProgress.nodes)}·nodes`
640 |     : '-'),
641 |     (aiInfoScoreElement.textContent = aiProgress ? formatDebugScore(aiProgress.score) : '-'),
642 |     !debug || 0 === debug.candidates.length)
643 | )
644 | return void aiCandidateListElement.replaceChildren();
645 | aiCandidateMetaElement.textContent = [
646 |     aiCandidateMetaElement.textContent,
647 |     `best·${formatDebugScore(debug.bestScore)}`,
648 |     `창·${Math.round(debug.windowCp)}`,
649 |     `p·${debug.ticketPower.toFixed(2)}`,
650 | ].join('·');
651 | const fragment = document.createDocumentFragment();
652 | for (const candidate of debug.candidates) fragment.appendChild(createAiCandidateRow(candidate));
653 | aiCandidateListElement.replaceChildren(fragment);
654 | }
655 | function createAiCandidateRow(candidate) {
656 |     const row = document.createElement('div');
657 |     ((row.className = 'ai-candidate-row'),
658 |     row.classList.toggle('is-selected', candidate.selected),
659 |     row.classList.toggle('is-principal', candidate.principal),
660 |     row.setAttribute('role', 'row'));
661 |     const sanCell = createAiCandidateCell('ai-candidate-san', candidate.san);
662 |     return (
663 |     candidate.selected && sanCell.appendChild(createAiCandidateBadge('선택')),
664 |     candidate.principal && sanCell.appendChild(createAiCandidateBadge('최선')),
665 |     row.append(
666 |     sanCell,
667 |     createAiCandidateCell('ai-candidate-score', formatDebugScore(candidate.score)),
668 |     createAiCandidateCell(
669 |     'ai-candidate-weight',
670 |     (function formatDebugWeight(weight) {
671 |         return 0 === weight
672 |         ? '0'
673 |         : weight < 0.001
674 |         ? '<0.001'
675 |         : weight.toFixed(3).replace(/0+$/, '').replace(/\./, '');
676 |     }))(candidate.tickets),
677 |     ),
678 |     createAiCandidateCell(
679 |     'ai-candidate-probability',
680 |     (function formatDebugProbability(probability) {
681 |         const percent = 100 * probability;
682 |         return 0 === percent ? '0%' : percent < 0.1 ? '<0.1%' : `${percent.toFixed(1)}%`;
683 |     }))(candidate.probability),
684 |     ),
685 |     ),
686 |     row
687 | );
688 | }
689 | function createAiCandidateCell(className, text) {
690 |     const cell = document.createElement('span');
691 |     return (
692 |     (cell.className = className),
693 |     cell.setAttribute('role', 'cell'),
694 |     (cell.textContent = text),
695 |     cell
696 | );
697 | }
698 | function createAiCandidateBadge(text) {
699 |     const badge = document.createElement('span');
700 |     return ((badge.className = 'ai-candidate-badge'), (badge.textContent = text), badge);

```

```

701 | }
702 | function getSqaureClasses(square, lastMove) {
703 |   const classes = ['board-square', 'light' === game.squareColor(square) ? 'is-light' : 'is-dark'],
704 |   destinationMove = legalMoves.find((move) => move.to === square);
705 |   return (
706 |     selectedSquare === square && classes.push('is-selected'),
707 |     destinationMove && classes.push(destinationMove.isCapture() ? 'is-capture' : 'is-legal'),
708 |     !lastMove ||
709 |     (lastMove.from !== square && lastMove.to !== square) ||
710 |     classes.push('is-last-move'),
711 |     (function isCheckedKing(square) {
712 |       const piece = game.get(square);
713 |       return Boolean(
714 |         piece && 'k' === piece.type && piece.color === game.turn() && game.isCheck(),
715 |         );
716 |     })(square) && classes.push('is-check'),
717 |     classes.join(' ')
718 |   );
719 | }
720 | function getBoardOrientation() {
721 |   return 'ai' === mode ? humanColor : game.turn();
722 | }
723 | function canUseBoard() {
724 |   return !(
725 |     game.isGameOver() ||
726 |     null !== timedOutColor ||
727 |     aiThinking ||
728 |     pendingPromotion ||
729 |     ('local' !== mode && game.turn() !== humanColor)
730 |   );
731 | }
732 | (boardElement.addEventListener('click', (event) => {
733 |   const button = (function getSqaureButton(target, boardElement) {
734 |     if (!(target instanceof Element)) return null;
735 |     const button = target.closest('.board-square');
736 |     return button && boardElement.contains(button) ? button : null;
737 |   })(event.target, boardElement);
738 |   button &&
739 |     (function isSquare(value) {
740 |       return 'string' === typeof value && SQUARE_SET.has(value);
741 |     })(button.dataset.square) &&
742 |     (function handleSquareClick(square) {
743 |       if (!canUseBoard()) return;
744 |       const piece = game.get(square);
745 |       if (selectedSquare) {
746 |         const movesToDestination = legalMoves.filter((move) => move.to === square);
747 |         return movesToDestination.length > 0
748 |           ? movesToDestination.some((move) => move.isPromotion())
749 |           ? ((pendingPromotion = {
750 |               from: selectedSquare,
751 |               to: square,
752 |               moves: movesToDestination,
753 |             })),
754 |           void render()
755 |           : void playMove({ from: selectedSquare, to: square })
756 |           : piece?.color === game.turn()
757 |           ? void selectSquare(square)
758 |           : (clearSelection(), void render());
759 |       }
760 |       piece?.color === game.turn() && selectSquare(square);
761 |     })(button.dataset.square);
762 |   }),
763 |   modeButtons.forEach((button) => {
764 |     button.addEventListener('click', () => {
765 |       const nextMode = button.dataset.mode;
766 |       (function isGameMode(value) {
767 |         return 'local' === value || 'ai' === value;
768 |       })(nextMode) &&
769 |         nextMode !== mode &&
770 |         ((mode = nextMode), resetGame());
771 |     });

```



```

772 |     },
773 |     colorButtons.forEach((button) => {
774 |         button.addEventListener('click', () => {
775 |             const nextColor = button.dataset.color;
776 |             (function isColor(value) {
777 |                 return 'w' === value || 'b' === value;
778 |             })(nextColor) &&
779 |                 nextColor !== humanColor &&
780 |                 ((humanColor = nextColor), resetGame());
781 |         });
782 |     },
783 |     presetButtons.forEach((button) => {
784 |         button.addEventListener('click', () => {
785 |             const preset = button.dataset.preset;
786 |             (function isAiPreset(value) {
787 |                 return 'easy' === value || 'normal' === value || 'hard' === value;
788 |             })(preset) && updateAiSettings({ ... AI_PRESETS[preset] });
789 |         });
790 |     },
791 |     clockPresetButtons.forEach((button) => {
792 |         button.addEventListener('click', () => {
793 |             const preset = button.dataset.clockPreset;
794 |             (function isClockPreset(value) {
795 |                 return (
796 |                     '1/1' === value ||
797 |                     '3/2' === value ||
798 |                     '5/3' === value ||
799 |                     '10/5' === value ||
800 |                     '15/10' === value
801 |                 );
802 |             })(preset) &&
803 |                 preset !== clockPreset &&
804 |                 ((clockPreset = preset), resetGame());
805 |         });
806 |     },
807 |     aiDepthInput.addEventListener('input', () => {
808 |         var value;
809 |         updateAiSettings({
810 |             ... aiSettings,
811 |             maxDepth:
812 |                 ((value = aiDepthInput.valueAsNumber), Math.min(5, Math.max(1, Math.round(value)))),
813 |         });
814 |     },
815 |     aiTimeInput.addEventListener('input', () => {
816 |         updateAiSettings({ ... aiSettings, timeLimitMs: aiTimeInput.valueAsNumber });
817 |     },
818 |     newGameButton.addEventListener('click', resetGame),
819 |     undoButton.addEventListener('click', function undoLastTurn() {
820 |         (cancelAiMove(),
821 |         (pendingPromotion = null),
822 |         0 !== game.history().length
823 |         ? ('ai' === mode && game.turn() === humanColor ? (game.undo(), game.undo()) : game.undo(),
824 |         clearSelection(),
825 |         resetClockState(),
826 |         restartClockForCurrentTurn(),
827 |         render())
828 |         : render());
829 |     },
830 |     promotionChoices.addEventListener('click', (event) => {
831 |         const target = event.target;
832 |         if (!(target instanceof Element)) return;
833 |         const button = target.closest('[data-promotion]'),
834 |             promotion = button ? dataset.promotion;
835 |         (function isPromotionPiece(value) {
836 |             return 'q' === value || 'r' === value || 'b' === value || 'n' === value;
837 |         })(promotion) &&
838 |             (function finishPromotion(promotion) {
839 |                 if (!pendingPromotion) return;
840 |                 if (!pendingPromotion.moves.find((move) => move.promotion === promotion)) return;
841 |                 const { from, to } = pendingPromotion;
842 |                 ((pendingPromotion = null), playMove({ from, to, promotion }));

```

```

843 |         .})(promotion);
844 |     },
845 |     resetClockState(),
846 |     restartClockForCurrentTurn(),
847 |     render());
848 | }());
849 |

```

==== style.css (14083 B / 3bb92984b4f4) ====

```

1 | *,
2 | *:before,
3 | *:after {
4 |     box-sizing: border-box;
5 | }
6 | html {
7 |     min-width: 1280px;
8 |     min-height: 100%;
9 |     background: var(--page);
10 |     color-scheme: light;
11 | }
12 | body {
13 |     min-width: 1280px;
14 |     min-height: 100vh;
15 |     min-height: 100dvh;
16 |     margin: 0;
17 |     background: var(--page);
18 |     font-family:
19 |         Inter,
20 |         ui-sans-serif,
21 |         system-ui,
22 |         -apple-system,
23 |         BlinkMacSystemFont,
24 |         Segoe UI,
25 |         sans-serif;
26 |     text-rendering: geometricPrecision;
27 | }
28 | button {
29 |     font: inherit;
30 | }
31 | button:not(:disabled) {
32 |     cursor: pointer;
33 | }
34 | button:disabled {
35 |     cursor: not-allowed;
36 | }
37 | :root {
38 |     --app-padding: 28px;
39 |     --viewport-height: 100vh;
40 |     --page: oklch(95% 0.02 85deg / 1);
41 |     --ink: oklch(26% 0.01 250deg / 1);
42 |     --muted: oklch(54% 0.01 185deg / 1);
43 |     --line: oklch(81% 0.02 80deg / 1);
44 |     --panel: oklch(99% 0.01 85deg / 1);
45 |     --button: oklch(33% 0.03 205deg / 1);
46 |     --button-text: oklch(100% 0 0deg / 1);
47 |     --accent: oklch(55% 0.15 35deg / 1);
48 |     --light-square: oklch(89% 0.05 90deg / 1);
49 |     --dark-square: oklch(59% 0.08 150deg / 1);
50 |     --last-move: oklch(80% 0.12 90deg / 1);
51 |     --legal: oklch(43% 0.12 260deg / 1);
52 |     --capture: oklch(52% 0.15 25deg / 1);
53 |     --shadow: 0 18px 42px oklch(24% 0.01 75deg / 0.18);
54 |     --clear-color: oklch(0% 0 0deg / 0);
55 |     --panel-soft: oklch(99% 0.01 85deg / 0.68);
56 |     --panel-raised: oklch(99% 0.01 85deg / 0.74);
57 |     --panel-strong: oklch(99% 0.01 85deg / 0.88);
58 |     --board-edge: oklch(33% 0.03 155deg / 1);
59 |     --last-move-strong: oklch(80% 0.12 90deg / 0.76);
60 |     --last-move-soft: oklch(80% 0.12 90deg / 0.68);
61 |     --legal-marker: oklch(43% 0.12 260deg / 0.42);
62 |     --capture-marker: oklch(52% 0.15 25deg / 0.74);

```

```

63 | --piece-shadow: oklch(0% 0 0deg / 0.2);
64 | --active-ring: oklch(100% 0 0deg / 0.3);
65 | --active-halo: oklch(33% 0.03 205deg / 0.18);
66 | --warning: oklch(69% 0.15 75deg / 1);
67 | --line-soft: oklch(81% 0.02 80deg / 0.58);
68 | --line-firm: oklch(81% 0.02 80deg / 0.62);
69 | --accent-soft: oklch(55% 0.15 35deg / 0.12);
70 | --button-soft: oklch(33% 0.03 205deg / 0.08);
71 | --overlay: oklch(21% 0.01 240deg / 0.58);
72 | }
73 | @supports (height: 100dvh) {
74 |   :root {
75 |     --viewport-height: 100dvh;
76 |   }
77 | }
78 | [hidden] {
79 |   display: none !important;
80 | }
81 | .chess-app {
82 |   --board-min-size: 560px;
83 |   --board-vertical-breathing: 104px;
84 |   --side-columns-width: 628px;
85 |   --grid-gaps-width: 30px;
86 |   --board-size: max(
87 |     var(--board-min-size),
88 |     min(
89 |       calc(
90 |         var(--viewport-height) - var(--app-padding) - var(--app-padding) -
91 |         var(--board-vertical-breathing)
92 |       ),
93 |       calc(
94 |         100vw - var(--app-padding) - var(--app-padding) - var(--side-columns-width) -
95 |         var(--grid-gaps-width)
96 |       )
97 |     )
98 |   );
99 |   --panel-max-height: calc(var(--viewport-height) - var(--app-padding) - var(--app-padding));
100 |   width: 100%;
101 |   min-width: 1280px;
102 |   min-height: var(--viewport-height);
103 |   display: grid;
104 |   grid-template-columns: 230px var(--board-size) 118px 280px;
105 |   gap: 10px;
106 |   align-items: stretch;
107 |   justify-content: center;
108 |   overflow: hidden;
109 |   padding: var(--app-padding);
110 |   background: var(--page);
111 |   color: var(--ink);
112 | }
113 | .board-stage {
114 |   width: var(--board-size);
115 |   align-self: center;
116 |   justify-self: center;
117 | }
118 | .clock-panel {
119 |   width: 100%;
120 |   height: var(--board-size);
121 |   min-width: 0;
122 |   min-height: 0;
123 |   align-self: center;
124 |   justify-self: stretch;
125 |   display: flex;
126 | }
127 | .chess-board {
128 |   width: 100%;
129 |   aspect-ratio: 1;
130 |   display: grid;
131 |   grid-template-columns: repeat(8, minmax(0, 1fr));
132 |   overflow: hidden;
133 |   border: 3px solid var(--board-edge);

```

```

134 |   border-radius: 8px;
135 |   box-shadow: var(--shadow);
136 |   background: var(--board-edge);
137 | }
138 | .board-square {
139 |   position: relative;
140 |   width: 100%;
141 |   aspect-ratio: 1;
142 |   display: grid;
143 |   place-items: center;
144 |   min-width: 0;
145 |   padding: 0;
146 |   border: 0;
147 |   color: var(--ink);
148 |   transition:
149 |     box-shadow 0.14s ease,
150 |     filter 0.14s ease,
151 |     transform 0.14s ease;
152 | }
153 | .board-square:not(:disabled):hover {
154 |   filter: brightness(1.05);
155 | }
156 | .board-square.is-light {
157 |   background: var(--light-square);
158 | }
159 | .board-square.is-dark {
160 |   background: var(--dark-square);
161 | }
162 | .board-square.is-selected {
163 |   box-shadow: inset 0 0 0 4px var(--legal);
164 | }
165 | .board-square.is-last-move {
166 |   box-shadow: inset 0 0 0 4px var(--last-move-strong);
167 | }
168 | .board-square.is-selected.is-last-move {
169 |   box-shadow:
170 |     inset 0 0 0 4px var(--legal),
171 |     inset 0 0 0 8px var(--last-move-soft);
172 | }
173 | .board-square.is-legal:after,
174 | .board-square.is-capture:after {
175 |   position: absolute;
176 |   z-index: 1;
177 |   content: '';
178 |   border-radius: 999px;
179 |   pointer-events: none;
180 | }
181 | .board-square.is-legal:after {
182 |   width: 28%;
183 |   height: 28%;
184 |   background: var(--legal-marker);
185 | }
186 | .board-square.is-capture:after {
187 |   inset: 12%;
188 |   border: 3px solid var(--capture-marker);
189 | }
190 | .board-square.is-check {
191 |   box-shadow: inset 0 0 0 5px var(--capture);
192 | }
193 | .chess-board.is-locked .board-square {
194 |   filter: saturate(0.86);
195 | }
196 | .piece {
197 |   z-index: 2;
198 |   display: block;
199 |   font-size: 4rem;
200 |   font-size: max(4rem, var(--board-size) / 8 * 0.92);
201 |   line-height: 1;
202 |   text-shadow: 0 2px 6px var(--piece-shadow);
203 |   -webkit-user-select: none;
204 |   user-select: none;

```

```

205 | }
206 | .coordinate-{
207 |   position: absolute;
208 |   z-index: 3;
209 |   font-size: 0.7rem;
210 |   font-size: max(0.7rem, var(--board-size) / 80);
211 |   font-weight: 800;
212 |   line-height: 1;
213 |   opacity: 0.72;
214 |   pointer-events: none;
215 | }
216 | .coordinate-rank-{
217 |   top: 6px;
218 |   left: 7px;
219 | }
220 | .coordinate-file-{
221 |   right: 7px;
222 |   bottom: 6px;
223 | }
224 | .control-panel,
225 | .info-panel-{
226 |   min-width: 0;
227 |   min-height: 0;
228 |   max-height: var(--panel-max-height);
229 |   display: flex;
230 |   flex-direction: column;
231 |   gap: 14px;
232 |   overflow: auto;
233 |   overscroll-behavior: contain;
234 |   scrollbar-gutter: stable;
235 | }
236 | .control-panel-{
237 |   justify-self: end;
238 | }
239 | .info-panel-{
240 |   justify-self: start;
241 | }
242 | .control-label-{
243 |   margin: 0;
244 |   color: var(--muted);
245 |   font-size: 0.78rem;
246 |   font-weight: 800;
247 |   letter-spacing: 0;
248 |   text-transform: uppercase;
249 | }
250 | .control-group,
251 | .status-panel,
252 | .ai-candidate-panel,
253 | .history-panel-{
254 |   display: grid;
255 |   gap: 10px;
256 |   padding-top: 16px;
257 |   border-top: 1px solid var(--line);
258 | }
259 | .history-panel-{
260 |   min-height: 0;
261 |   flex: 1;
262 |   grid-template-rows: auto minmax(0, 1fr);
263 | }
264 | .segmented-control-{
265 |   display: grid;
266 |   grid-template-columns: repeat(2, minmax(0, 1fr));
267 |   gap: 4px;
268 |   padding: 4px;
269 |   border: 1px solid var(--line);
270 |   border-radius: 8px;
271 |   background: var(--panel-soft);
272 | }
273 | .segmented-control-three-{
274 |   grid-template-columns: repeat(3, minmax(0, 1fr));
275 | }

```

```

276 | .clock-preset-control {
277 |     display: grid;
278 |     grid-template-columns: repeat(5, minmax(0, 1fr));
279 |     gap: 4px;
280 |     padding: 4px;
281 |     border: 1px solid var(--line);
282 |     border-radius: 8px;
283 |     background: var(--panel-soft);
284 | }
285 | .segmented-button,
286 | .primary-button,
287 | .ghost-button,
288 | .promotion-button {
289 |     min-height: 42px;
290 |     border-radius: 6px;
291 |     font-weight: 800;
292 |     letter-spacing: 0;
293 | }
294 | .segmented-button {
295 |     border: 0;
296 |     background: var(--clear-color);
297 |     color: var(--muted);
298 | }
299 | .segmented-button.is-active {
300 |     background: var(--button);
301 |     color: var(--button-text);
302 | }
303 | .clock-settings .segmented-button {
304 |     min-height: 36px;
305 |     padding-inline: 4px;
306 |     font-size: 0.82rem;
307 | }
308 | .ai-settings {
309 |     gap: 12px;
310 | }
311 | .range-control,
312 | .toggle-control {
313 |     display: grid;
314 |     gap: 8px;
315 |     min-width: 0;
316 |     color: var(--ink);
317 |     font-size: 0.92rem;
318 |     font-weight: 800;
319 | }
320 | .range-control span,
321 | .toggle-control {
322 |     align-items: center;
323 | }
324 | .range-control span {
325 |     display: flex;
326 |     justify-content: space-between;
327 |     gap: 10px;
328 | }
329 | .range-control strong {
330 |     color: var(--accent);
331 |     white-space: nowrap;
332 | }
333 | .range-control input[type='range'] {
334 |     width: 100%;
335 |     accent-color: var(--button);
336 | }
337 | .toggle-control {
338 |     grid-template-columns: auto minmax(0, 1fr);
339 |     padding: 10px;
340 |     border: 1px solid var(--line);
341 |     border-radius: 8px;
342 |     background: var(--panel-raised);
343 | }
344 | .toggle-control input {
345 |     width: 18px;
346 |     height: 18px;

```

```

347 |   accent-color: var(--button);
348 | }
349 | .action-row {
350 |   display: grid;
351 |   grid-template-columns: 1fr 1fr;
352 |   gap: 10px;
353 | }
354 | .primary-button,
355 | .ghost-button {
356 |   border: 1px solid var(--button);
357 | }
358 | .primary-button {
359 |   background: var(--button);
360 |   color: var(--button-text);
361 | }
362 | .ghost-button {
363 |   background: var(--clear-color);
364 |   color: var(--button);
365 | }
366 | .primary-button:disabled,
367 | .ghost-button:disabled,
368 | .segmented-button:disabled {
369 |   opacity: 0.46;
370 | }
371 | .game-status {
372 |   min-height: 1.4em;
373 |   color: var(--accent);
374 |   font-size: 1.35rem;
375 |   line-height: 1.2;
376 | }
377 | .clock-display {
378 |   width: 100%;
379 |   height: 100%;
380 |   min-height: 0;
381 |   min-width: 0;
382 |   display: flex;
383 |   flex-direction: column;
384 |   gap: 10px;
385 |   justify-content: space-between;
386 | }
387 | .clock-card {
388 |   --clock-bg: var(--panel-raised);
389 |   --clock-border: var(--line);
390 |   --clock-side: var(--muted);
391 |   --clock-time: var(--ink);
392 |   width: 100%;
393 |   min-block-size: 82px;
394 |   max-block-size: 110px;
395 |   min-width: 0;
396 |   display: grid;
397 |   place-content: center;
398 |   gap: 4px;
399 |   padding: 12px 8px;
400 |   border: 1px solid var(--clock-border);
401 |   border-radius: 8px;
402 |   background: var(--clock-bg);
403 |   text-align: center;
404 | }
405 | #white-clock {
406 |   --clock-bg: oklch(99% 0.01 85deg / 1);
407 |   --clock-border: oklch(81% 0.02 80deg / 1);
408 |   --clock-side: oklch(54% 0.01 185deg / 1);
409 |   --clock-time: oklch(26% 0.01 250deg / 1);
410 | }
411 | #black-clock {
412 |   --clock-bg: oklch(33% 0.03 205deg / 1);
413 |   --clock-border: oklch(25% 0.02 210deg / 1);
414 |   --clock-side: oklch(99% 0.01 85deg / 0.74);
415 |   --clock-time: oklch(99% 0.01 85deg / 1);
416 | }
417 | .clock-card.is-active {

```

```

418 | border-color: var(--button);
419 | box-shadow:
420 |   inset 0 0 0 2px var(--active-ring),
421 |   0 0 0 2px var(--active-halo);
422 | }
423 | .clock-card.is-warning .clock-time {
424 |   color: var(--warning);
425 | }
426 | .clock-card.is-danger .clock-time,
427 | .clock-card.is-timeout .clock-time {
428 |   color: var(--capture);
429 | }
430 | .clock-side {
431 |   color: var(--clock-side);
432 |   font-size: 0.72rem;
433 |   font-weight: 800;
434 |   line-height: 1;
435 | }
436 | .clock-time {
437 |   font-variant-numeric: tabular-nums;
438 |   color: var(--clock-time);
439 |   font-size: 1.35rem;
440 |   line-height: 1;
441 | }
442 | .status-grid {
443 |   display: grid;
444 |   grid-template-columns: 1fr;
445 |   gap: 8px;
446 |   margin: 0;
447 | }
448 | .status-grid div {
449 |   min-width: 0;
450 |   padding: 10px;
451 |   border: 1px solid var(--line);
452 |   border-radius: 8px;
453 |   background: var(--panel-raised);
454 | }
455 | .status-grid dt,
456 | .status-grid dd {
457 |   margin: 0;
458 | }
459 | .status-grid dt {
460 |   color: var(--muted);
461 |   font-size: 0.72rem;
462 |   font-weight: 800;
463 | }
464 | .status-grid dd {
465 |   margin-top: 4px;
466 |   overflow-wrap: anywhere;
467 |   font-size: 0.92rem;
468 |   font-weight: 800;
469 | }
470 | .history-header {
471 |   display: flex;
472 |   align-items: center;
473 |   justify-content: space-between;
474 | }
475 | .ai-candidate-panel {
476 |   min-height: 0;
477 | }
478 | .ai-info-grid {
479 |   display: grid;
480 |   grid-template-columns: repeat(2, minmax(0, 1fr));
481 |   gap: 6px;
482 |   margin: 0;
483 | }
484 | .ai-info-grid div {
485 |   min-width: 0;
486 |   padding: 8px;
487 |   border: 1px solid var(--line);
488 |   border-radius: 8px;

```



```

489 | background: var(--panel-raised);
490 | }
491 | .ai-info-grid dt,
492 | .ai-info-grid dd {
493 | margin: 0;
494 | }
495 | .ai-info-grid dt {
496 | color: var(--muted);
497 | font-size: 0.64rem;
498 | font-weight: 800;
499 | text-transform: uppercase;
500 | }
501 | .ai-info-grid dd {
502 | margin-top: 3px;
503 | overflow: hidden;
504 | font-size: 0.8rem;
505 | font-weight: 800;
506 | line-height: 1.1;
507 | text-overflow: ellipsis;
508 | white-space: nowrap;
509 | }
510 | .ai-info-grid dd[id='ai-info-nodes'],
511 | .ai-info-grid dd[id='ai-info-score'],
512 | .ai-info-grid dd[id='ai-info-depth'] {
513 | font-variant-numeric: tabular-nums;
514 | }
515 | .ai-candidate-header {
516 | display: flex;
517 | align-items: baseline;
518 | justify-content: space-between;
519 | gap: 8px;
520 | }
521 | .ai-candidate-meta {
522 | min-width: 0;
523 | overflow: hidden;
524 | color: var(--muted);
525 | font-size: 0.68rem;
526 | font-weight: 800;
527 | text-align: right;
528 | text-overflow: ellipsis;
529 | white-space: nowrap;
530 | }
531 | .ai-candidate-table {
532 | min-height: 0;
533 | overflow: hidden;
534 | border: 1px solid var(--line);
535 | border-radius: 8px;
536 | background: var(--panel-raised);
537 | }
538 | .ai-candidate-list {
539 | max-height: 172px;
540 | overflow: auto;
541 | }
542 | .ai-candidate-row {
543 | display: grid;
544 | grid-template-columns: minmax(3rem, 1.3fr) 2.6rem 3.2rem 3.2rem;
545 | gap: 6px;
546 | align-items: center;
547 | min-width: 0;
548 | padding: 6px 8px;
549 | border-bottom: 1px solid var(--line-soft);
550 | font-size: 0.72rem;
551 | font-weight: 800;
552 | line-height: 1.15;
553 | }
554 | .ai-candidate-row:last-child {
555 | border-bottom: 0;
556 | }
557 | .ai-candidate-head {
558 | color: var(--muted);
559 | background: var(--panel-strong);

```

```

560 |   font-size: 0.66rem;
561 |   text-transform: uppercase;
562 | }
563 | .ai-candidate-row.is-selected {
564 |   background: var(--accent-soft);
565 | }
566 | .ai-candidate-row.is-principal:not(.is-selected) {
567 |   background: var(--button-soft);
568 | }
569 | .ai-candidate-san,
570 | .ai-candidate-score,
571 | .ai-candidate-weight,
572 | .ai-candidate-probability {
573 |   min-width: 0;
574 |   overflow: hidden;
575 |   text-overflow: ellipsis;
576 |   white-space: nowrap;
577 | }
578 | .ai-candidate-score,
579 | .ai-candidate-weight,
580 | .ai-candidate-probability {
581 |   text-align: right;
582 |   font-variant-numeric: tabular-nums;
583 | }
584 | .ai-candidate-badge {
585 |   display: inline-block;
586 |   margin-left: 4px;
587 |   padding: 1px 3px;
588 |   border-radius: 4px;
589 |   background: var(--button);
590 |   color: var(--button-text);
591 |   font-size: 0.56rem;
592 |   line-height: 1;
593 |   vertical-align: 1px;
594 | }
595 | .move-list {
596 |   min-height: 0;
597 |   max-height: none;
598 |   overflow: auto;
599 |   border: 1px solid var(--line);
600 |   border-radius: 8px;
601 |   background: var(--panel-raised);
602 | }
603 | .move-row {
604 |   display: grid;
605 |   grid-template-columns: 3rem minmax(0, 1fr) minmax(0, 1fr);
606 |   gap: 8px;
607 |   padding: 8px 10px;
608 |   border-bottom: 1px solid var(--line-firm);
609 |   font-weight: 800;
610 | }
611 | .move-row:last-child {
612 |   border-bottom: 0;
613 | }
614 | .move-number {
615 |   color: var(--muted);
616 | }
617 | .move-san {
618 |   min-width: 0;
619 |   overflow-wrap: anywhere;
620 | }
621 | .move-empty {
622 |   margin: 0;
623 |   padding: 14px 12px;
624 |   color: var(--muted);
625 |   font-weight: 700;
626 | }
627 | .promotion-overlay {
628 |   position: fixed;
629 |   inset: 0;
630 |   z-index: 10;

```

```
631 |     display: grid;
632 |     place-items: center;
633 |     padding: 20px;
634 |     background: var(--overlay);
635 | }
636 | .promotion-overlay[hidden]{
637 |     display: none;
638 | }
639 | .promotion-dialog{
640 |     width: 360px;
641 |     display: grid;
642 |     gap: 16px;
643 |     padding: 20px;
644 |     border: 1px solid var(--line);
645 |     border-radius: 8px;
646 |     background: var(--panel);
647 |     box-shadow: var(--shadow);
648 | }
649 | .promotion-dialog h2{
650 |     margin: 0;
651 |     font-size: 1.1rem;
652 | }
653 | .promotion-choices{
654 |     display: grid;
655 |     grid-template-columns: repeat(4, minmax(0, 1fr));
656 |     gap: 8px;
657 | }
658 | .promotion-button{
659 |     min-height: 70px;
660 |     border: 1px solid var(--button);
661 |     background: var(--button-text);
662 |     color: var(--ink);
663 |     font-size: 2.4rem;
664 |     line-height: 1;
665 | }
666 |
```